

TupperDMX Simulator

User manual · audio, MIDI and DMX on one timeline

Version 1.0.0

Built 2026-08-15

Contents

1. Introduction

- 1.1 What it does
- 1.2 The five words you need
- 1.3 Requirements
- 1.4 Installing and running
- 1.5 Where things are stored
- 1.6 About this manual

2. Operating modes

- 2.1 Mode 1 — BBeat and the Arduino controller
- 2.2 Mode 2 — the laptop on stage
- 2.3 Mode 3 — headless in the rack, run from a tablet
- 2.4 Choosing

3. The main interface

- 3.1 The window title
- 3.2 Resizing the zones
- 3.3 The stage overlay buttons
- 3.4 The position display
- 3.5 Keyboard shortcuts
- 3.6 The ☰ menu

4. The stage view

- 4.1 What you are looking at
- 4.2 Entering the designer
- 4.3 The edit toolbar
- 4.4 Placing fixtures
- 4.5 The Inspector
- 4.6 The stage in its own window

5. The Add fixture dialog

- 5.1 The library
- 5.2 The detail tabs
- 5.3 The footer — the patch decision
- 5.4 The Scenes tab

6. The timeline

- 6.1 The transport bar
- 6.2 What a clip is
- 6.3 Anatomy
- 6.4 Undo and redo
- 6.5 Selecting
- 6.6 Moving, resizing and fading
- 6.7 Adding clips
- 6.8 Context menus
- 6.9 The clip editor

7. The top toolbar

7.1 SHOW — the project
7.2 SETLIST — the songs
7.3 SONG FILES — the current song's media
7.4 VERSIONS — history
7.5 What is saved where

8. The clip library and the DMX monitor

8.1 The Library tab
8.2 The DMX Channels tab

9. Settings

9.1 Input / Output
9.2 Web Remote
9.3 Audio
9.4 Virtual stage
9.5 Logging
9.6 Resetting

10. The web remote

10.1 Turning it on
10.2 The panel
10.3 Everything routes normally
10.4 Rack mode

11. A show from start to finish

Step 1 — Create the show
Step 2 — Load the first backing track
Step 3 — Get the tempo and the offset right
Step 4 — Patch the rig
Step 5 — Build the looks you will reuse
Step 6 — Write the lighting
Step 7 — Count-ins and cues for the band
Step 8 — Set up the audio for the stage
Step 9 — Route the lights
Step 10 — Set up the tablet
Step 11 — Rehearse, then pin a snapshot
Step 12 — Take it to the gig
Quick reference

Appendix A. MIDI note reference

A.1 The full layout
A.2 Song parts (notes 61 – 69)
A.3 Colour macros (notes 72 – 75)
A.4 The START palette (note 60)
A.5 Things that catch people out
A.6 One universal layout, many fixtures

Appendix B. Building a rack image with Buildroot

B.1 What you need
B.2 Lay the two trees out side by side
B.3 Build

B.4 Write it to the target
B.5 What ends up on the box
B.6 The file share
B.7 Running it
B.8 USB DMX and USB MIDI
B.9 Changing the configuration
B.10 Other boards
B.11 Known gaps

1. Introduction

TupperDMX Simulator is a lighting-control application for live bands. It plays a backing track, runs a click for the drummer, fires DMX lighting cues from a MIDI timeline, and draws the result on a virtual stage — all from one clock, so the lights land on the beat whether or not any lighting hardware is plugged in.

It is built for the band that runs its own show: you design the lighting at home against a virtual rig, carry the whole project to the venue in one folder, and run the night from a tablet.

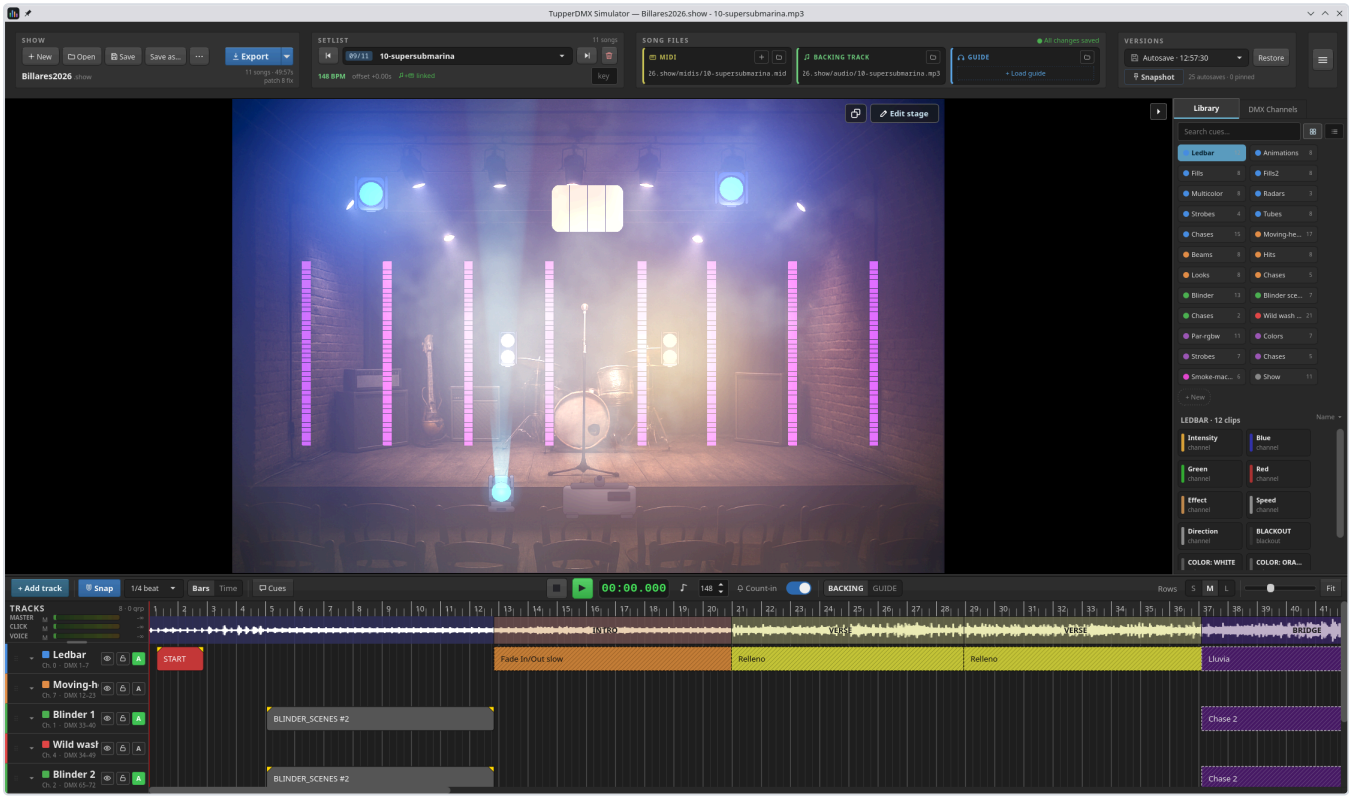


Figure 1 — *The main window, mid-design: a lit virtual stage above the clip timeline, with the show toolbar on top and the clip library on the right.*

1.1 What it does

Audio and click. One stereo output carries three buses — the backing track, the metronome click and a spoken voice guide. Each has its own volume and mute. The click can be sent to one side and the backing to the other, so the musicians hear the click on in-ears while the room hears only the music.

MIDI-driven lighting. Each song owns a MIDI file, edited in the app as *clips* on a timeline — one lane per patched fixture. A clip is not a note in the musical sense: it is an instruction to a light.

A virtual stage. Every patched fixture is drawn in real time from the DMX that is actually being sent, with beams, haze, smoke and — on machines with a usable OpenGL driver — a high-dynamic-range renderer with volumetric fog. What you design at the kitchen table is what the rig does at the venue.

A software DMX controller. A pure-Python controller emulating a hardware lighting desk is built in. With the factory settings it runs in-process and feeds the stage directly, so **the app is fully usable with no hardware**

at all — no interface, no MIDI cable, no lights. When there *is* a rig, the same controller drives it over Art-Net or a USB DMX interface.

A tablet remote. An optional web page on the local network shows the setlist, the current section, a visual metronome, a scrubbable waveform and a grid of manual light cues. It is how the show is actually run from the stage.

1.2 The five words you need

The rest of this manual leans on five terms. They are worth ten seconds each.

Show — a project. A show is a *folder* named `MyBand2026.show/` holding a `show.json` manifest, the MIDI files it owns, any audio that came in with it, show-local fixture definitions and a `versions/` history. Opening a show replaces the setlist and the patch. One folder is the whole gig.

Song — one entry in the setlist. A song carries a backing track, a MIDI file, a BPM, a time offset, an optional musical key, an optional count-in and an optional *guide* track (a full-mix reference used for design that is never played live).

Fixture and patch — a fixture is one light. *Patching* it means giving it a DMX start address, a MIDI channel and a place on the stage. The patch lives in the show, so two shows can use completely different rigs.

Clip — one instruction on one fixture's timeline lane. Its MIDI **note** is a *verb* (set a channel, load a scene, run a chase, black out), its MIDI **channel** selects **which fixture** the verb applies to, and its **velocity** is that verb's parameter. That is the whole model, and everything in the timeline follows from it.

Scene and bank — a saved look for one fixture type: a set of channel values with a name. Scenes live in *banks* (`FILLS`, `STROBES`, `BEAMS`…) so a MIDI verb can address them by number. Scenes belong to the fixture **type**, so every show that uses that type shares them.

1.3 Requirements

Operating system	Linux, Windows or macOS
Python	3.13 or newer (only when running from source)
Audio	any output device the system exposes
Lighting hardware	**none required** — optional Art-Net node or USB DMX interface
MIDI hardware	**none required** — optional MIDI output port
Graphics	any; the HDR stage renderer additionally wants a working OpenGL driver

The spoken count-in and voice cues need a text-to-speech voice, downloaded on demand (about 60 MB, once) from **Settings ▸ Audio**. Everything else works without it.

1.4 Installing and running

From a release build, run the executable. From source, dependencies are managed with uv:

```
git clone https://github.com/lothwen/tupperDMX-simulator.git
cd tupperDMX-simulator
uv sync                    # install dependencies, create .venv
uv run python main.py      # run
```

Rack mode

```
uv run python main.py --headless
```

Headless mode runs everything except the desktop window: the software controller, DMX and MIDI output, playback and the web remote. It is meant for a small box in the rack with no screen, where **the web remote is the only interface**. It force-enables the remote in memory without writing that choice to your preferences, and never downloads a TTS voice.

`--headless` is the only flag the app defines; anything else on the command line is handed to Qt, so `-platform`, `-style` and friends still work.

1.5 Where things are stored

Two locations, with a strict division of labour.

The show folder holds everything about *the gig*: the setlist, each song's files and settings, the fixture patch and the web-remote cues. It is self-contained and portable — copy the folder, or export a `.showpack` bundle. By default new shows are created under `Documents/TupperDMX/`.

The preferences file holds everything about *this machine*: audio device, I/O routing, log level, window sizes, and whether this box has a GPU worth using for the stage. It never contains show data, so moving a laptop between venues never drags a setlist along with it.

Platform	Preferences
Linux	`~/.config/tupperDMX/userprefs.json`
Windows	`%APPDATA%\tupperDMX\userprefs.json`
macOS	`~/Library/Application Support/tupperDMX/userprefs.json`

Deleting `userprefs.json` resets the application to factory settings. **It does not touch any show** — your projects are separate folders with their own version history.

1.6 About this manual

Chapter 2 lays out the three ways this software gets used, because which one you are in decides how much of the rest matters. Chapters 3 to 10 then describe the interface surface by surface, and chapter 11 walks a complete show from an empty project to a gig, cross-referencing the rest. Appendix A is the raw MIDI note layout, for when you need to read or write the notes directly; appendix B builds the bootable image for a rack box.

Every screenshot in this manual was taken from the running application against a real setlist, using the capture script that lives beside these pages.

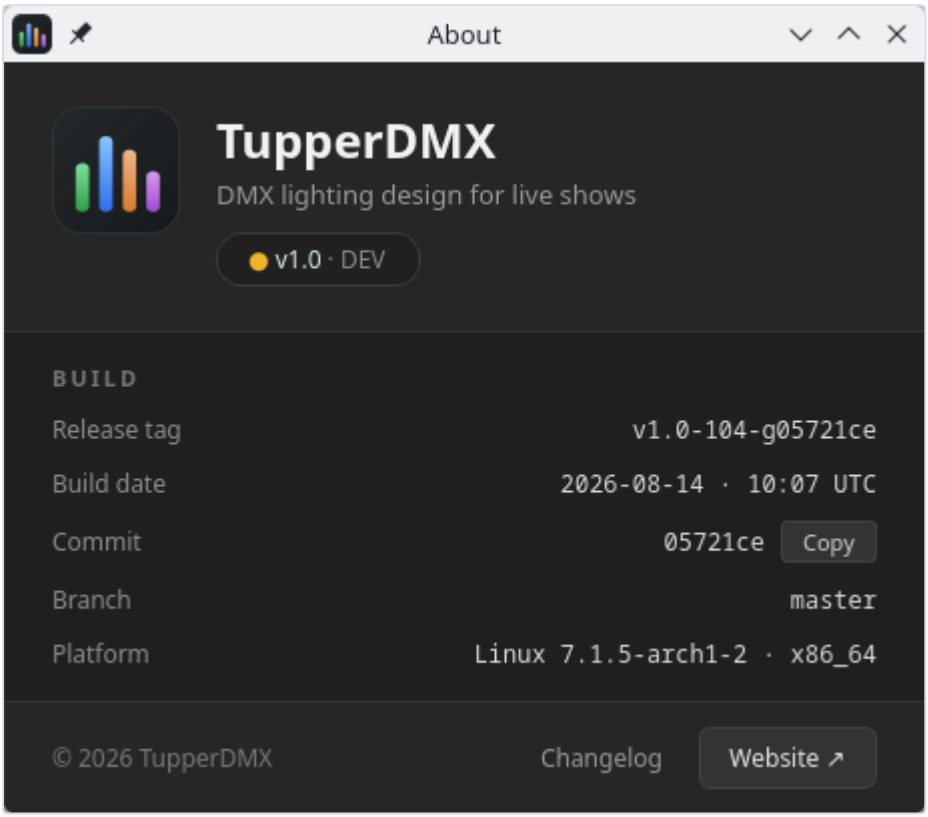


Figure 2 — *The About box: build provenance for bug reports — release tag, build date, commit, branch and platform.*

☰ ▸ **About TupperDMX** shows exactly which build you are running. The **Copy** button next to the commit puts the hash on the clipboard, which is the single most useful thing to include when reporting a problem.

2. Operating modes

The same show can be run three different ways. They differ in *which box plays the music*, *which box turns MIDI into DMX*, and *what the operator touches* — and that choice decides how much of this manual applies to you.

All three end in the same place: a DMX universe driving real fixtures. What changes is everything upstream of it.

	Plays the audio	Makes the DMX	Operator touches	Chapter
Mode 1	BBeat	tupperDMX-arduino	the player	the design chapters only
Mode 2	TupperDMX Simulator	TupperDMX Simulator	a laptop on stage	all of it
Mode 3	TupperDMX Simulator	TupperDMX Simulator	a tablet	all of it

2.1 Mode 1 — BBeat and the Arduino controller

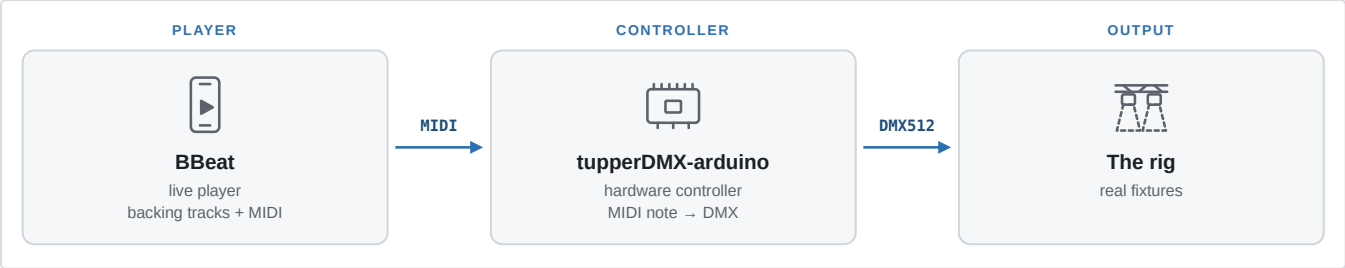


Figure 3 — *Mode 1: a live player sends MIDI to a hardware controller, which drives the rig.*

The band already runs a live player — **BBeat** — for its backing tracks and click. That player also sends the MIDI that the show's lighting is written in. A companion project, **tupperDMX-arduino**, receives those notes on a microcontroller and turns them into DMX for the rig.

Nothing in this chain is a computer running a desktop application. It is small, boots instantly, and has nothing to crash mid-set.

Where this application fits. It is the *design tool*. You build the fixture patch, write the lighting on the timeline and check the result on the virtual stage, then export the song's MIDI (**SHOW ▸ Export ▾ ▸ Export MIDI…**) and load that into BBeat. On the night, this software is not running at all.

> The internal software controller in this application is a port of the Arduino firmware's own logic, so a note that produces a look here produces the same look there. That is the whole point of designing against the simulator: the verbs, the scene banks and the channel layouts are shared.

What matters to you in this manual: the stage designer (4), the fixture browser (5), the timeline (6) and the clip library (8). The I/O routing page matters only for previewing while you work — leave it on **All software**.

2.2 Mode 2 — the laptop on stage

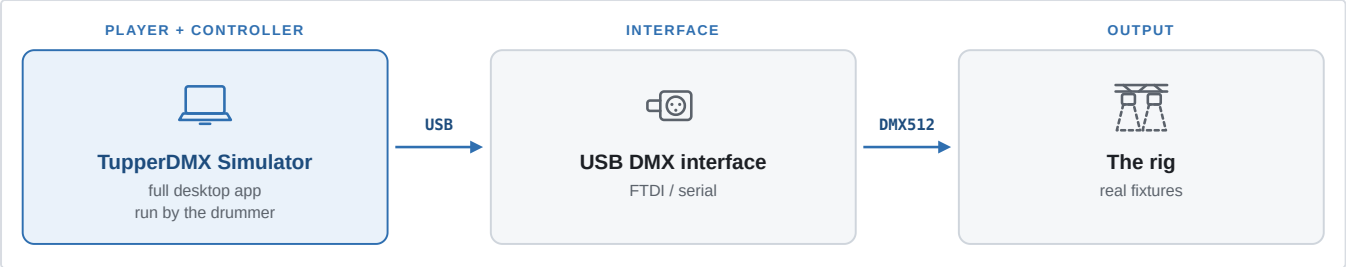


Figure 4 — *Mode 2: the full desktop application drives a USB DMX interface directly.*

One machine does everything. The full desktop application plays the backing track, runs the click, sequences the MIDI, turns it into DMX with its internal controller, and pushes that out of a USB DMX interface into the rig. Typically the laptop sits by the drummer, who starts each song.

This is the simplest chain to set up and the one with the most control during a show: the whole timeline, the stage and the DMX monitor are in front of you, and a fixture can be re-patched between songs.

It is also the chain with a laptop screen on stage, which is a decision about sightlines as much as about technology.

Setup: Settings ▸ Input / Output, quick mode **Software + real lights**, then tick **USB DMX** and pick the interface. Or use Art-Net instead if the rig is on a network node. → 9.1

What matters to you in this manual: all of it.

2.3 Mode 3 — headless in the rack, run from a tablet

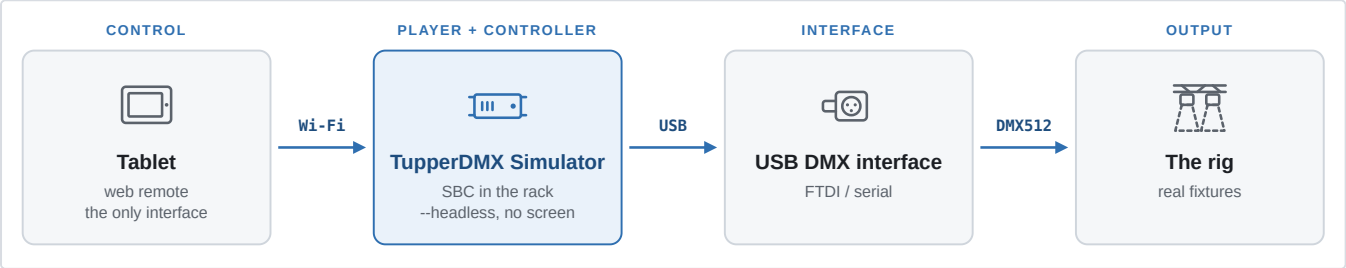


Figure 5 — *Mode 3: a tablet drives a headless machine in the rack over the network.*

The same application, on a small board in the rack, started with `--headless`. It has no window and needs no screen: it boots into the show, plays the audio, sequences the lighting and drives the USB DMX interface exactly as the laptop would. The **web remote** is the only interface, on a tablet on a mic stand.

This is the configuration for a band that does not want a laptop visible, and it is the most robust of the three on stage — nothing to knock, nothing to sleep, nothing to accidentally click.

The trade is that you cannot edit during the show. The remote gives you the setlist, transport, count-in, manual light cues and a blackout; it does not give you the timeline. Design happens beforehand, on a normal machine, and travels to the rack as a `.showpack` bundle.

```
uv run python main.py --headless
```

> Headless mode force-enables the web remote so there is a way in, and that switch lives only in memory — it never rewrites the preferences of the machine it is running on.

Setup: design on a laptop, **SHOW ▸ Export** a bundle, import it on the rack machine, set its I/O routing once, then start it headless. → 10.4, 7.1

The rack machine does not have to be a general-purpose computer. **Appendix B** builds a single bootable image that comes up straight into headless mode, with no login, no desktop and nothing else installed. → Appendix B

What matters to you in this manual: all of it for design, and chapter 10 for the night itself.

2.4 Choosing

- **Already running BBeat live?** Mode 1. Use this software to design and export, and change nothing else about your rig.
- **Want one box to do everything, and happy to have it on stage?** Mode 2.
- **Want the show run from a tablet with nothing else visible?** Mode 3.

Modes 2 and 3 are the same software with a different front end, so moving between them costs nothing: the show folder is identical, and only the I/O routing and the choice of `--headless` differ. Mode 1 shares the fixture definitions and the note vocabulary but not the runtime, so the hand-off there is the exported MIDI file.

3. The main interface

The window has four zones and no menu bar. Everything lives on the top toolbar, on the timeline's own toolbar, or behind the ☰ button in the top-right corner.

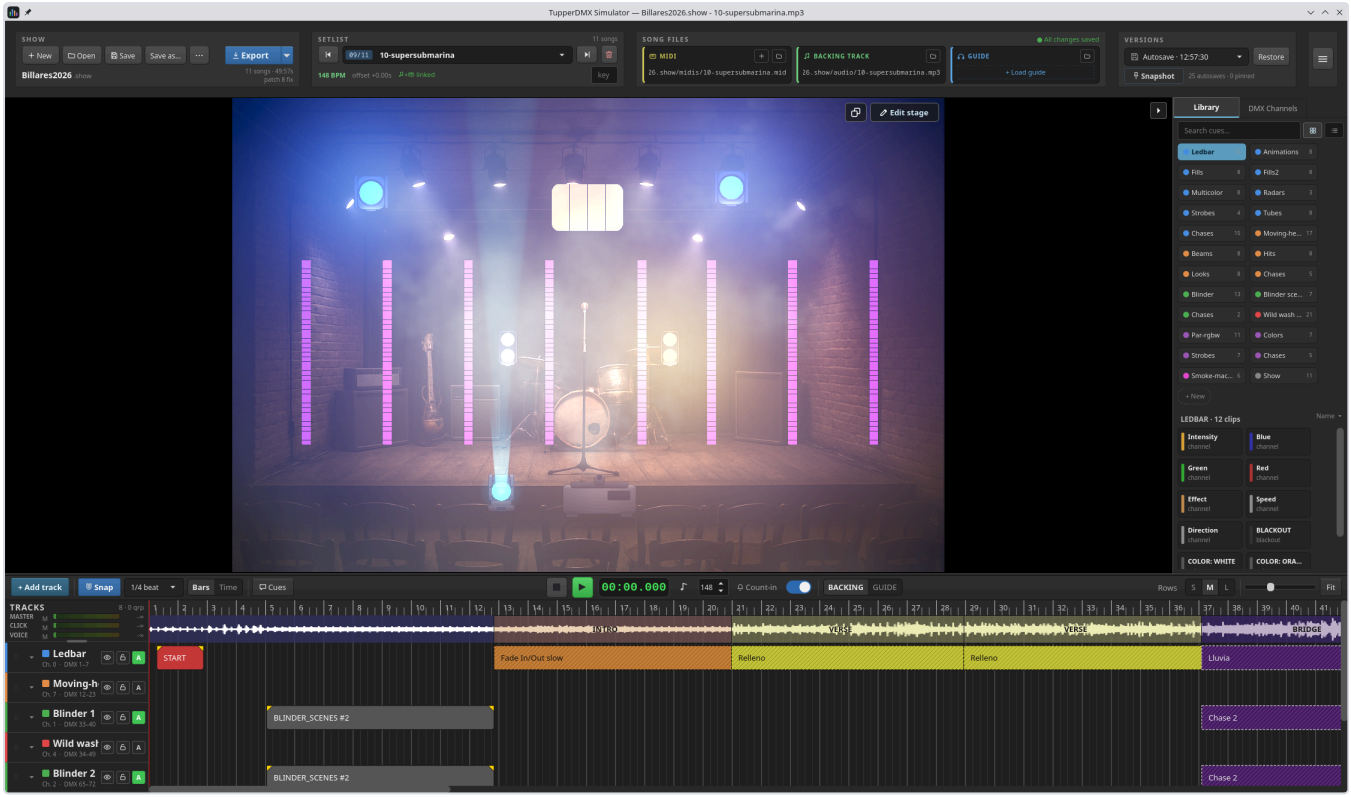


Figure 6 — *The four zones: the show toolbar across the top, the virtual stage, the right sidebar, and the timeline along the bottom.*

Zone	What it is	Chapter
Top toolbar	The show: project, setlist, song files, version history	7
Virtual stage	The rig, drawn live from the DMX being sent	4
Right sidebar	Two tabs — **Library** (draggable cues) and **DMX Channels**	8
Timeline	Transport bar over one lane per patched fixture	6

3.1 The window title

The title bar composes what is currently open:

```
TupperDMX Simulator — Billares2026.show - 10-supersubmarina.mp3
```

The show name appears once a project is open, the file name once the current song has a backing track. If you are ever unsure which project you are editing, this is the fastest place to look.

3.2 Resizing the zones

Two drag handles divide the window, and both remember where you left them.

- **Between the stage and the timeline** — a horizontal divider. Drag it up to give the timeline more lanes, down to make the stage bigger. Neither pane can be collapsed to nothing.
- **Between the stage and the sidebar** — a vertical divider.

The sidebar also has a one-click hide: the small **chevron** at the top-right corner of the stage slides it out of the way over about a fifth of a second and gives the whole width to the stage. The button's tooltip flips between **"Hide DMX panel"** and **"Show DMX panel"**, and the state is remembered between sessions.

3.3 The stage overlay buttons

Three buttons float over the stage picture rather than sitting in a toolbar, so they do not steal room from the rig.

Button	Position	What it does
Edit stage (pencil)	top-right of the stage image	Enters the stage designer — grid on, fixtures become draggable
Undock (two rectangles)	left of *Edit stage*	Moves the stage into its own **Virtual Stage** window, typically for a second display
Chevron	top-right of the viewport	Hides or shows the right sidebar

Once the stage is undocked, the timeline takes over the whole main window and its toolbar moves to the bottom edge. The stage window can be closed to dock it back; if it ends up on a display you no longer have, **☰ ▸ Dock stage** brings it home.

3.4 The position display

The transport's read-out, in the middle of the timeline toolbar, shows `MM:SS.mmm` while the song runs.

During a count-in it reads something different:

```
COUNT-IN −03.2
```

This is not a quirk of the display. **A count-in is negative timeline time.** Playback starts before zero, the playhead waits at the left edge, and audio, MIDI, lights and the metronome all begin at t = 0 together. Pausing during the count-in resets to zero; seeking cancels it. See 6.1.3.

3.5 Keyboard shortcuts

Key	Where	Action
Space	anywhere	Play / pause
Ctrl + Z	anywhere	Undo the last timeline edit
Ctrl + Shift + Z	anywhere	Redo
Delete	timeline	Replace the selected clips with blackouts
Ctrl + C / **Ctrl + V**	timeline	Copy the selection / paste it at the playhead
Ctrl + mouse wheel	timeline	Zoom horizontally, anchored under the pointer

Space is deliberately suppressed while a text field, a spin box or an editable drop-down has focus, so typing a song title never starts the show. It is also suppressed while a dialog is open.

Undo and redo have **no button and no menu entry**, and nothing greys out when there is nothing left to undo — the shortcuts are the whole interface. See 6.4.

3.6 The ☰ menu

The hamburger button at the far right of the top toolbar holds the three actions that do not belong to any one section:

Entry	Notes
Dock stage	Disabled unless the stage is floating
Settings…	Opens the preferences dialog — chapter 9
About TupperDMX	Version and build provenance

4. The stage view

The stage is not a diagram. It is a renderer fed by the DMX that is actually leaving the application, so a beam on screen is a beam the rig would throw.



Figure 7 — *The stage under a designed look: LED bars, blinders, a moving head cutting a shaft through haze, and a smoke machine feeding the air.*

It has two modes. Normally it just draws — nothing on it can be clicked, dragged or knocked out of place, which is what you want during a show. **Edit stage** turns it into the designer.

4.1 What you are looking at

Two things decide how a fixture is drawn, and they are decided separately.

Its category picks the body and the kind of light it makes. A moving head gets a yoke and a shaft; an LED bar gets a row of pixels; a wash panel gets a surface that glows without a visible shaft at all.

Its channels pick the shape of that light. Whatever capabilities the fixture's DMX channels declare — zoom, gobo, prism, frost, colour mixing, strobe — become layers that shape the beam. There is no per-fixture drawing code: a fixture the application has never seen before still renders from its definition.

Haze is what makes a beam visible. In clean air a light throws no shaft at all; you would see only the lens and the pool it lands in. The **HAZE** slider sets how much haze is in the air *before* any hazer or smoke machine in the patch runs — and any fog-producing fixture adds to it on top.

Fixture sizes on stage are deliberately **not** to scale. Every body is normalised to a similar size so that a beam and a glow read correctly next to each other; at true scale a scanner head sits inside its own beam and a large wash panel has no room to glow.

4.2 Entering the designer

Click **Edit stage**. The button disappears, a faint grid appears over the stage picture, and the edit toolbar drops into the top-left corner.

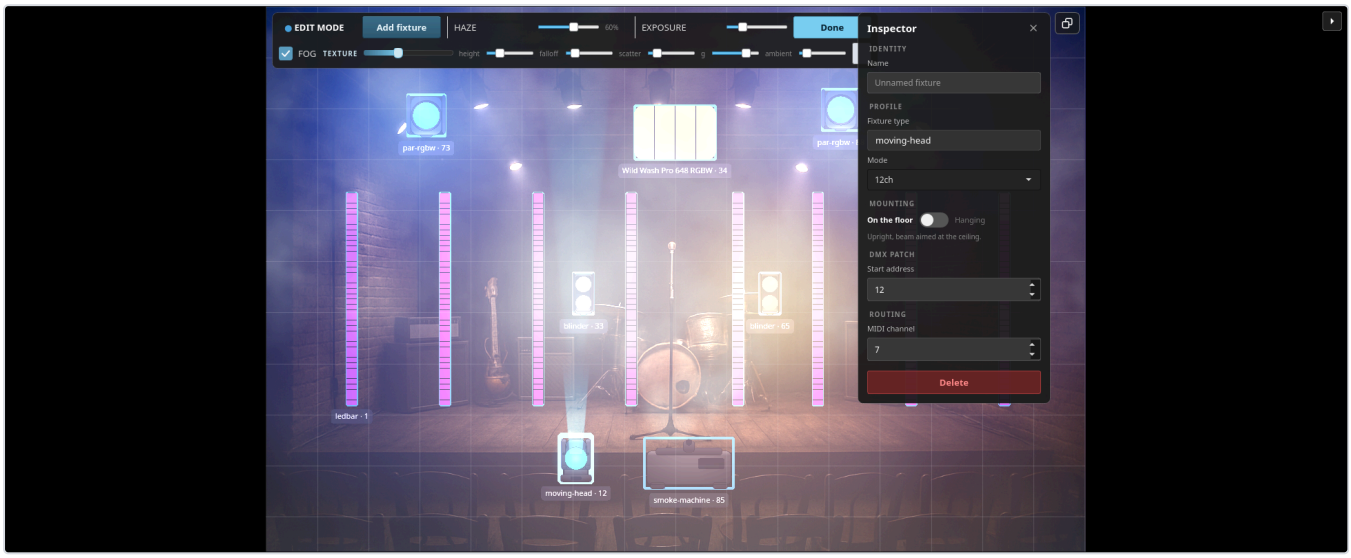


Figure 8 — *The stage designer: edit toolbar top-left, fixture labels under each body, and the Inspector for the selected moving head.*

The grid is a visual aid only — **the stage does not snap.** (The timeline does; the stage does not.)

4.3 The edit toolbar

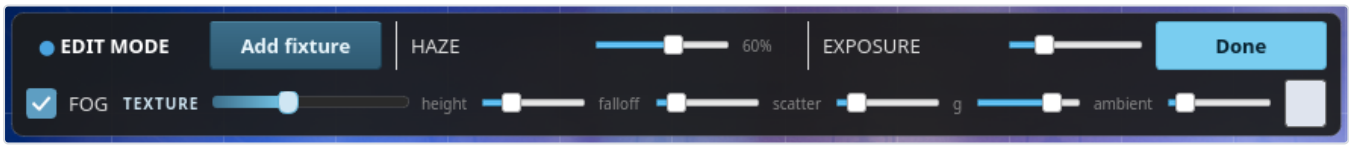


Figure 9 — *The first row of the edit toolbar: add a fixture, set the haze baseline, set the exposure, and leave.*

Control	What it does
● EDIT MODE	A label, so it is obvious the stage is live for editing
Add fixture	Opens the fixture browser — chapter 5
HAZE	0–100 %. How much haze is in the air before any hazer runs. The percentage beside it updates live
EXPOSURE	0.1× – 4.0×. How hard the rig is driven before highlights roll off to white. *Only present when the GPU renderer is running*
Done	Leaves edit mode

HAZE moves immediately. Unlike a real hazer, which drifts up over seconds, dragging the slider snaps the air straight to the new level — nobody adjusting a slider wants to wait twelve seconds to see what they did.

Done does not "apply" anything. Every edit in the designer — a drag, a rename, a new address, a deletion — is saved the moment you make it. *Done* only puts the stage back into playback mode.

The fog row



Figure 10 — *The fog row: the master switch and the six sliders that shape the air.*

The second row appears only when the GPU renderer is running. It shapes volumetric fog — instead of scaling each beam by one stage-wide haze number, the renderer marches every view ray through the air that is actually there, so a shaft brightens as it crosses a bank of smoke and god rays fall out on their own.

Control	What it does
FOG	The master switch. **While it is off, every slider beside it does nothing**
TEXTURE	How ragged the air is. At 0 the air is a perfectly smooth medium — the translucent-sheet look. Around 1 a pocket of air can be completely empty, which is what gives a shaft its billows. Higher still, the billows separate into banks with clear air between them: a stage under a smoke machine rather than under a hazer
height	How far up from the floor the fog reaches
falloff	How sharply it thins with height. Haze settles, so a slab of even density reads as wrong
scatter	How much of the light passing through comes back to you
g	Forward scattering. This is why a beam aimed at you is blinding and the same beam across the stage is a shaft
ambient	A stage-wide key on the air itself, so smoke is visible before any fixture is
swatch	What colour the air scatters

> **There is no fog density control here, and that is deliberate.** Density *is* the **HAZE** slider on the row above — the fog pass reads it as the amount of medium in clean air. The controls on this row shape that medium; they do not add more of it.

Turning the GPU renderer on and off is a separate, machine-level decision that lives in **Settings ▸ Virtual stage** (9.4) — quality settings there, appearance here.

4.4 Placing fixtures

A new fixture drops at the centre of the stage. Drag it where it belongs.

- The position is only visual while you drag; it is **written down when you let go**.
- A multi-body fixture — an eight-segment LED bar, say — moves **one body at a time**. That is how the bars get spread across the truss.
- Each fixture carries a label beneath it while editing, reading its name and DMX start address (`moving-head · 12`). On a multi-body fixture only the first body is labelled.

Where you drop it decides how it is rigged. Anything landing in the upper third of the picture is assumed to be flown; anything lower is assumed to stand on the deck. It is a starting guess, written down once and then left alone — a guess re-evaluated during the drag would flip a fixture over halfway across the stage. Correct it in the Inspector whenever the guess is wrong.

4.5 The Inspector

Select a fixture and the Inspector appears in the top-right corner. It is the whole per-fixture settings surface.

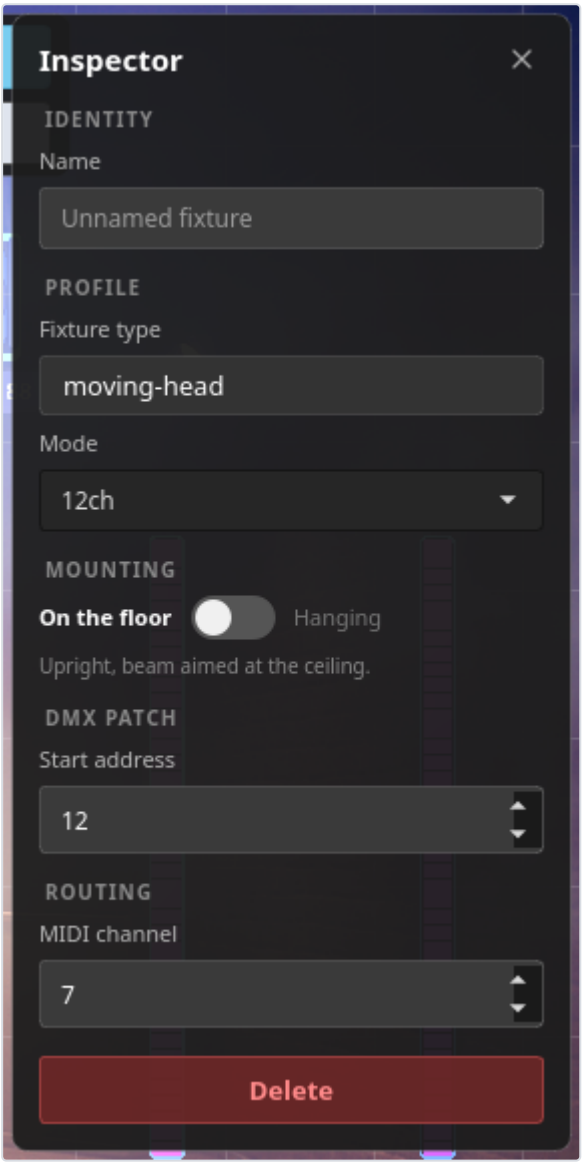


Figure 11 — *The Inspector for a moving head: identity, profile, mounting, patch and routing.*

Section	Field	Notes
IDENTITY	Name	Free text. Shows on the timeline lane and the DMX monitor. Blank falls back to the type name
PROFILE	Fixture type	Read-only. A type's DMX footprint is not editable from here
	Mode	**Only shown when the type declares more than one mode.** Changing it changes the channel layout, so the body is rebuilt
MOUNTING	On the floor / Hanging	**Only shown for fixtures that can be flown.** A smoke machine or a floor can always stands on its own feet, so it gets no switch
DMX PATCH	Start address	1–512
ROUTING	MIDI channel	0–15. See the warning below
	Delete	Asks for confirmation, and says how many bars go with it for a multi-body fixture

Mounting flips two things at once: the housing turns upside down, and the beam is aimed at the floor instead of the ceiling. The hint under the switch says which state you are in — *"Upside down, beam aimed at the stage."* or *"Upright, beam aimed at the ceiling."*

> **The MIDI channel is the fixture's identity as far as your clips are concerned.** Every clip on a timeline lane is bound to its fixture by that channel, not by name or address. Two fixtures may not share one: the app refuses with *"MIDI channel N is already used by another fixture"* and puts the old value back. Changing a channel on a fixture that already has clips moves those clips to whatever fixture now owns that channel — set it once, when you patch.

Closing the Inspector with the ✕ simply clears the selection; it does not undo anything.

4.6 The stage in its own window



Figure 12 — *The stage undocked into its own window, ready for a second display.*

Click the undock button and the stage becomes a separate **Virtual Stage** window. The main window hands its whole height to the timeline, and the sidebar moves in beside it.

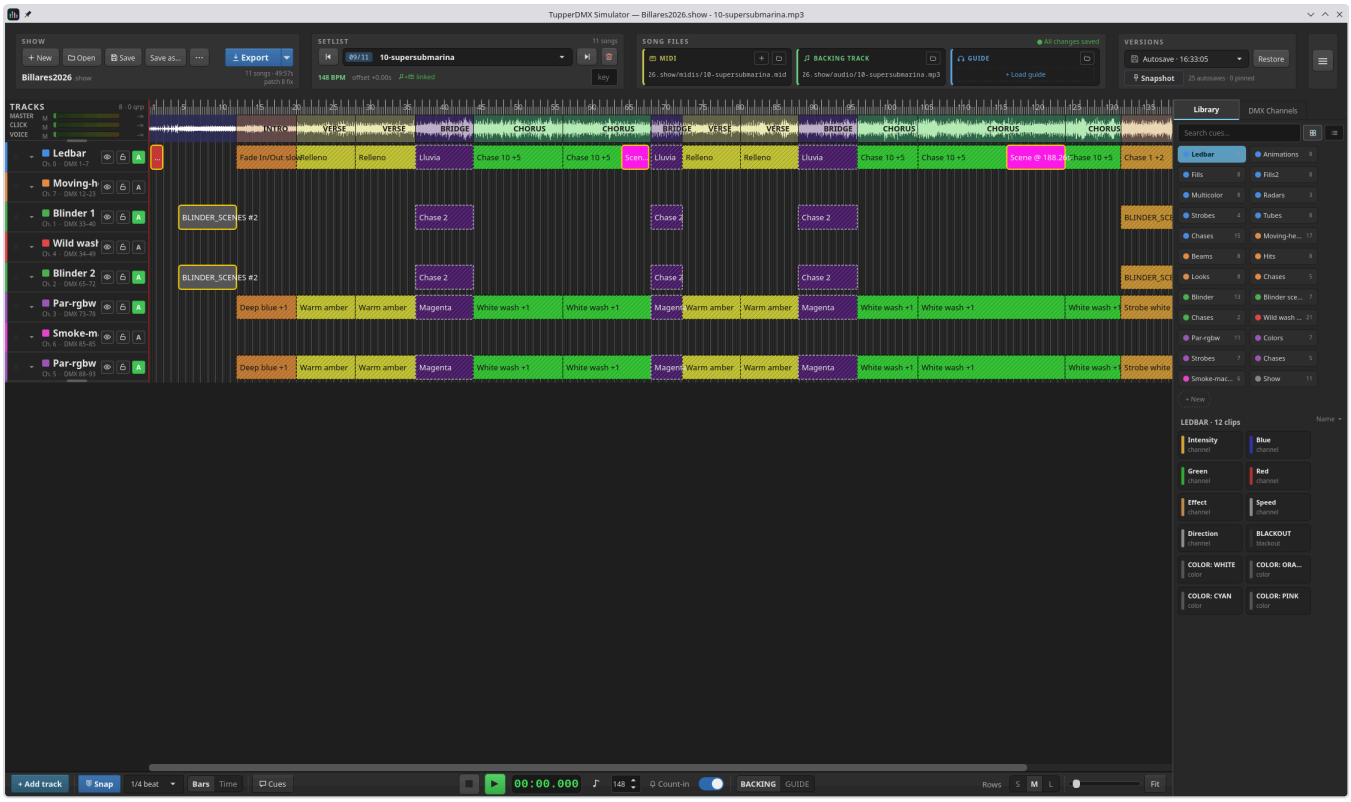


Figure 13 — *Undocked: the timeline owns the window, its toolbar moves to the bottom edge, and the library gets a full-height column.*

The stage cannot be closed into nowhere — closing the floating window docks it back into the main window. If it opens on a display that is no longer connected, ☰ ▸ **Dock stage** retrieves it.

5. The Add fixture dialog

Reached from **Edit stage ▸ Add fixture**, or straight from the timeline with **+ Add track** (a timeline lane *is* a patched fixture, so both open the same flow).

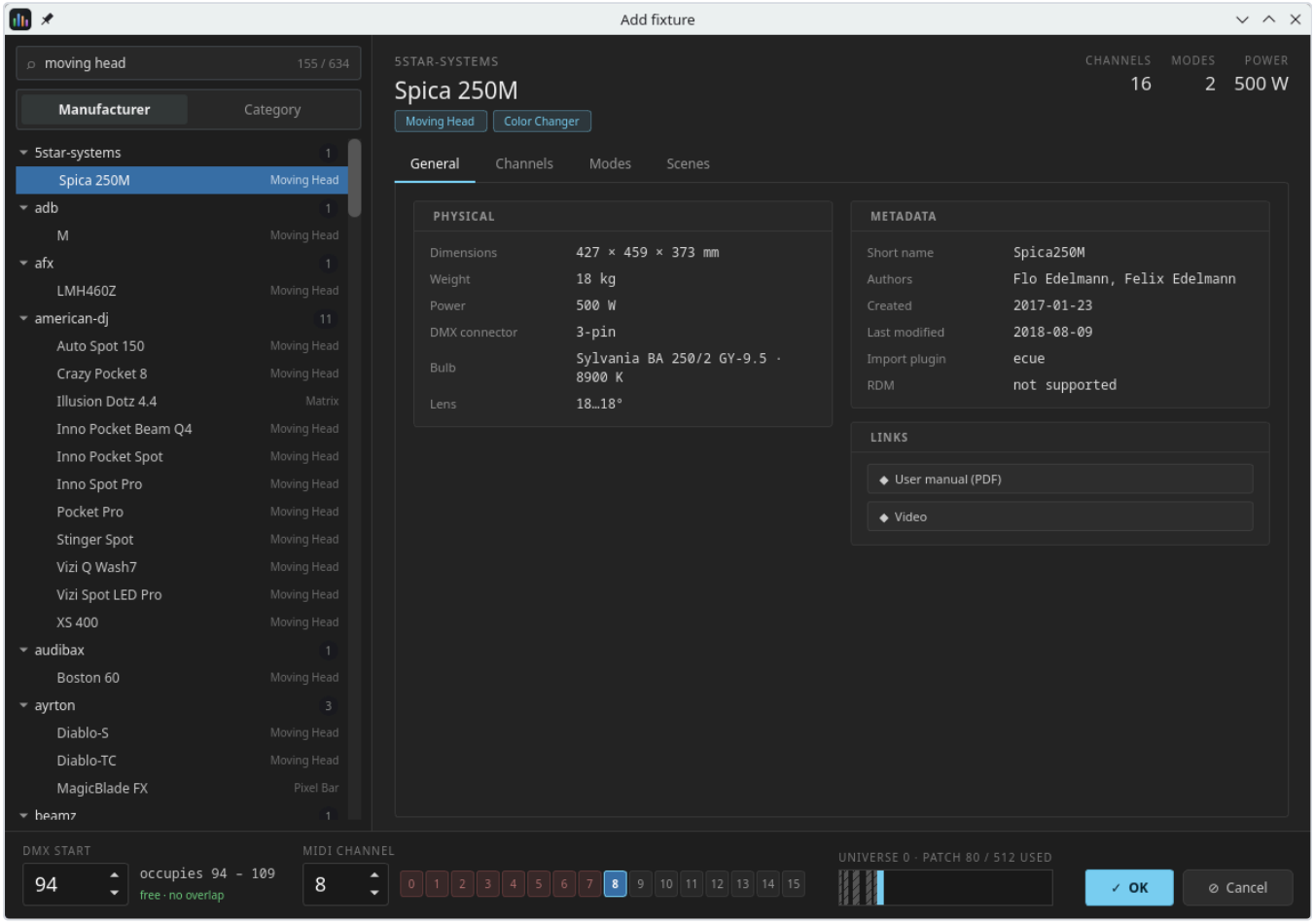


Figure 14 — *The fixture browser: library on the left, the selected fixture's detail tabs in the middle, and the patch decision along the bottom.*

The dialog has three regions, and they answer three different questions:

1. **the library**, on the left — *which fixture?*
2. **the detail tabs**, in the middle — *is this really the one?*
3. **the footer**, along the bottom — *where does it go?*

Only the footer changes your show. Everything else is a browser.

5.1 The library

The fixture library is the Open Fixture Library format, so any OFL definition works. Fixtures come from three layers, each overriding the one before: the definitions shipped with the application, the ones you have added under your preferences folder, and any that travel inside the open show.

Search (`Search fixtures, manufacturers…`) matches across manufacturers and fixture names, and the counter beside it shows how much of the library you are looking at — `155 / 634`. Groups start collapsed and expand automatically while you are searching.

The **Manufacturer | Category** segmented control changes how the tree is grouped. Whichever grouping you are *not* using appears as a dim caption on each row, so a manufacturer-grouped list still tells you that *Spica 250M* is a Moving Head. Fixtures with no manufacturer folder collect under **Generic**; uncategorised ones under **Uncategorised**.

5.2 The detail tabs

At the top of the panel: the manufacturer, the fixture name, its category chips, and three headline numbers — **CHANNELS**, **MODES**, **POWER**.

General

Up to four cards, each hidden when the definition has nothing to put in it:

- **PHYSICAL** — dimensions, weight, power draw, power and DMX connectors, bulb, lens.
- **METADATA** — short name, authors, created and last-modified dates, the import plugin the definition came through, and RDM support.
- **LINKS** — buttons that open the manufacturer's manual, product page or video in your system browser.
- **COMMENT** — whatever notes the definition carries.

Channels

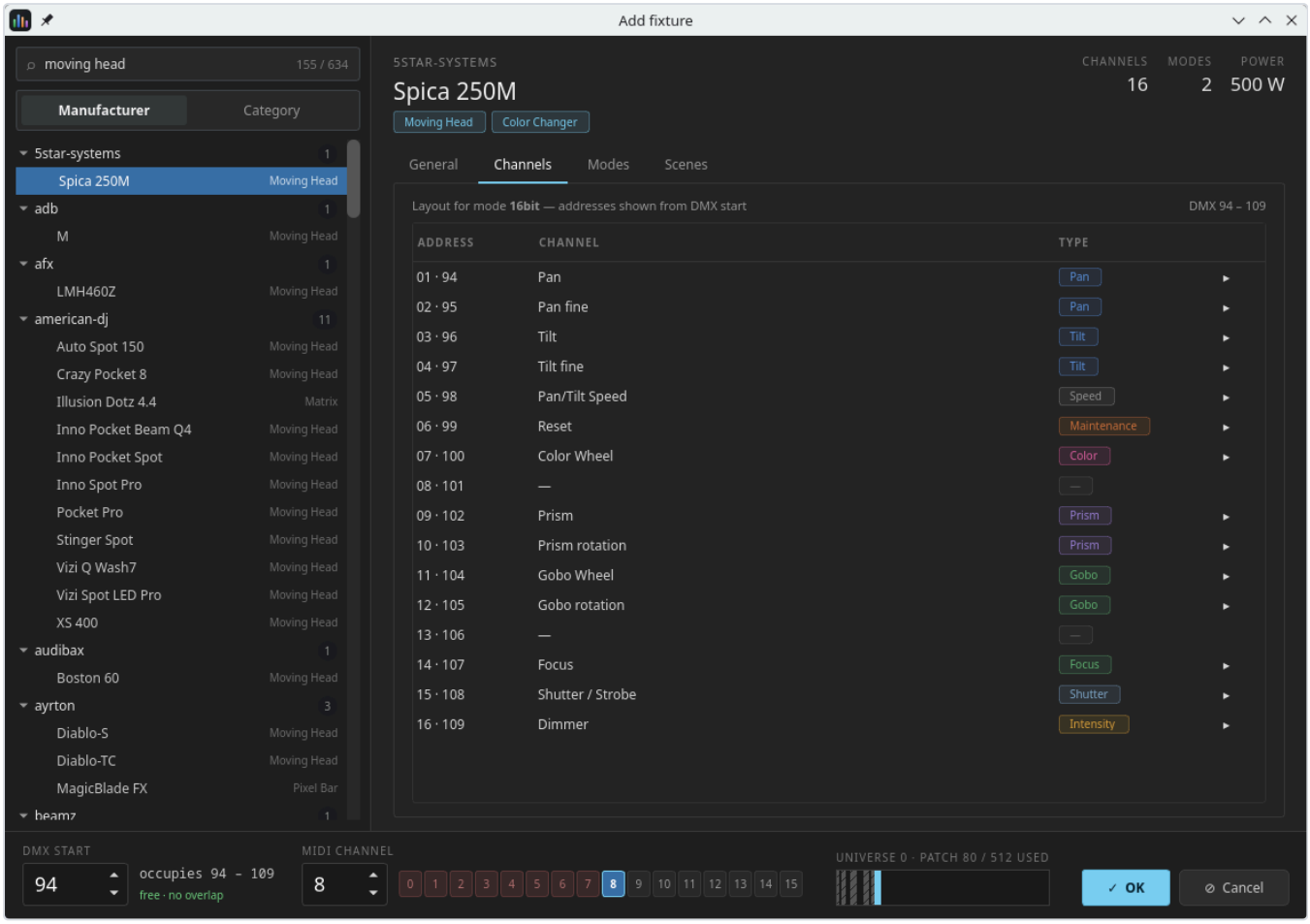


Figure 15 — *The Channels tab: every DMX slot of the selected mode, addressed from the chosen start.*

Every DMX slot of the currently selected mode, **addressed from the DMX start you have picked in the footer** — so the `ADDRESS` column reads `01 · 94`, the slot within the fixture and the real universe address side by side. The caption above the table names the mode, and the range appears on the right.

Each row carries the channel name and a colour-coded type chip (`Pan`, `Tilt`, `Intensity`, `Color`, `Gobo`, `Shutter`, `Maintenance` …). The ▸ on the right expands the row into that channel's capability ranges — the low and high DMX values, what the manufacturer calls each one, and its capability type.

A slot the parser could not resolve is shown as **—** and kept in place. It still consumes an address, which is what keeps the rest of the layout honest.

Modes

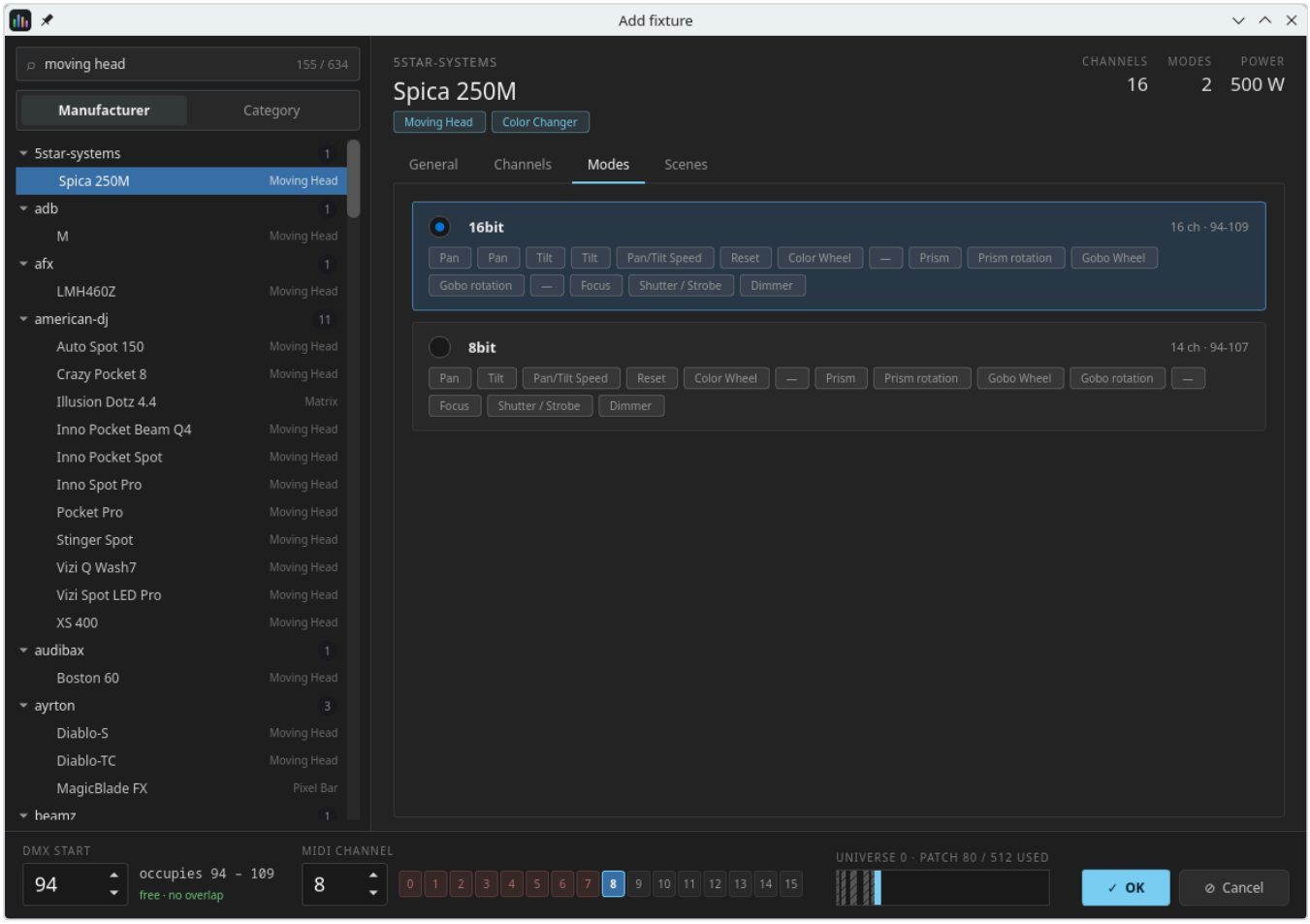


Figure 16 — *The Modes tab: one card per mode, with its footprint and its channel list.*

One card per mode: a radio button, the mode name, its short-name chip, its footprint (`16 ch · 94-109`), and a flow of every channel name in it. This is where you choose which mode to patch. The **Channels** tab follows your choice.

Pick the mode the fixture is actually set to. A moving head in its 8-channel mode patched here as 16-channel will respond to the wrong addresses for everything after the first few.

Scenes

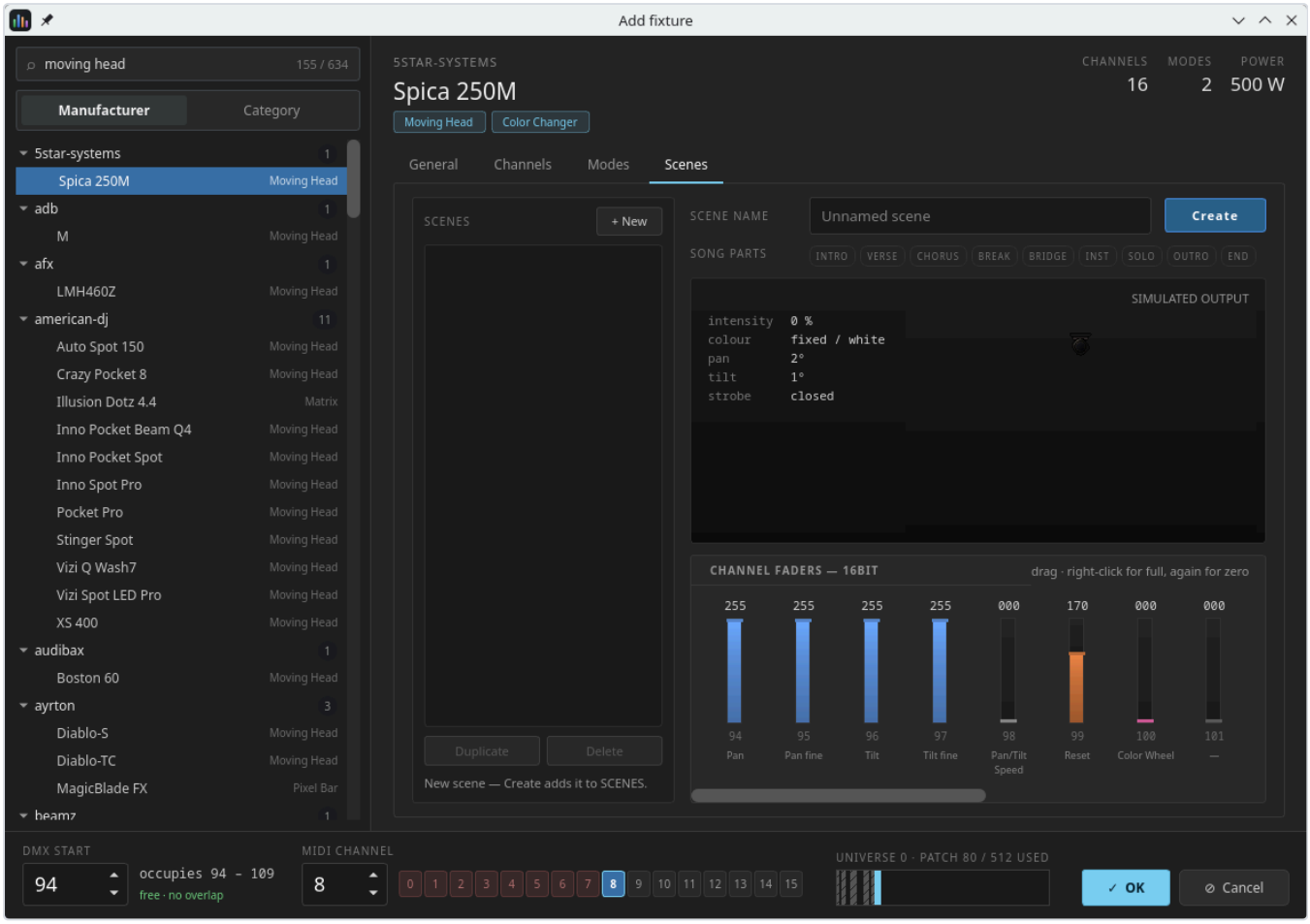


Figure 17 — *The Scenes tab: banks and scenes for this fixture type, with the console that dials them in.*

This tab edits stored looks for the fixture **type** — see 5.4.

5.3 The footer — the patch decision



Figure 18 — *The footer refusing an illegal patch: the address overlaps an existing fixture, so OK is disabled.*

Control	What it tells you
DMX START	The address, 1–512. Beside it, `occupies 94 – 109` and the status: `free · no overlap`, `overlaps Ledbar 1`, or `needs 16 ch · past 512`
MIDI CHANNEL	The channel, 0–15, with all sixteen as clickable buttons. Channels already in use are marked, and picking one says `already used`
PatchBar	The whole universe as a strip. Hatched blocks are already patched, the solid blue block is where this fixture would land — **and it turns red when it collides**
UNIVERSE 0 · PATCH n / 512 USED	How much of the universe the show is using
✓ OK / ⊘ Cancel	**OK stays disabled until the patch is legal**

You cannot create an illegal patch from this dialog. The Inspector applies the same two checks afterwards, so you cannot edit your way into one either.

5.4 The Scenes tab

A *scene* is a named set of channel values for a fixture type, stored in a *bank*. Banks and scene numbers are what the MIDI verbs address, which is why they are numbered rather than just named.

Left panel. A tree of banks with their scenes beneath. Each scene shows a colour swatch painted by the real renderer and, if it has any, abbreviated song-part tags. **+ New** offers **New scene** and **New bank…**; **Duplicate** and **Delete** work on whatever is selected. Banks and scenes are renamed in place by clicking the name, and reordered by dragging.

Right panel. The scene name, a row of nine song-part chips (INTRO, VERSE, CHORUS, BREAK, BRIDGE, INST, SOLO, OUTRO, END — tag a scene and the app will offer it automatically during that part of a song), the **SIMULATED OUTPUT** card, and the channel faders.

SIMULATED OUTPUT is the stage's own renderer fed the values you are dialling, next to a read-out of what they resolve to — intensity, colour, pan and tilt, strobe, effect, zoom. It is not an approximation of the stage; it *is* the stage.

CHANNEL FADERS is a lighting console. Press anywhere on a fader and the value jumps there, then follows the drag. **Right-click flashes a channel to full; right-click again returns it to zero.** A channel whose capabilities name discrete options gets a ▾ beside its name — click it and those options appear as a grid of live tiles (animated, when the definition ships artwork for them).

The workflow is: dial a look → type a name → **Create**. Selecting an existing scene loads it and the button becomes **Save**. The status line under the tree always says where you stand — *"New scene — Create adds it to SCENES."*, *"Unsaved changes — press Save to write them to the fixture's .tdmx.json."*, or a count of what is stored.

Two things about this tab are worth knowing before you use it.

It is not covered by OK and Cancel. This tab edits the fixture *library*, which every show using that type shares. **Create** and **Save** write to disk immediately. Cancelling the dialog does not undo them. (If the definition sits somewhere the app cannot write — inside the application bundle — it is forked into your own fixtures folder first, and your copy wins from then on.)

A saved song binds a scene by its position, not its name. MIDI verbs address "scene 3 of bank 1". Renaming a scene or a bank is safe. **Inserting, deleting or reordering scenes shifts the numbering, and existing songs will load a different look.** Add new scenes at the end of a bank, or in a new bank, once a show depends on them.

6. The timeline

The timeline is where a song's lighting is written. It shows the song's audio as a waveform, its arrangement as labelled sections, and its lighting as *clips* on one lane per patched fixture.

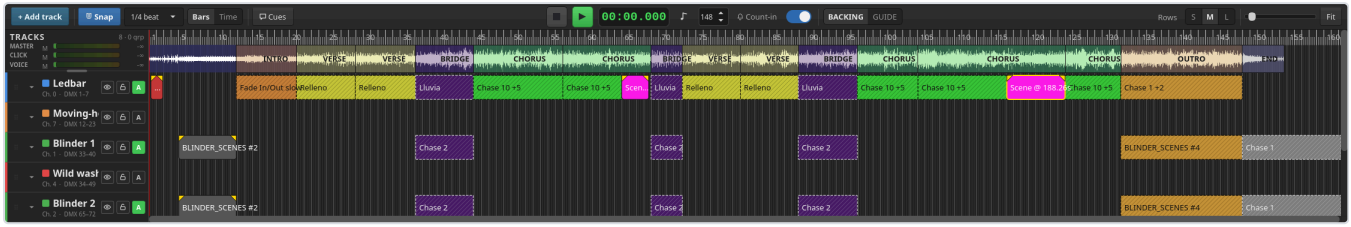


Figure 19 — *The timeline at song scale: waveform with arrangement sections on top, one lane per fixture below.*

6.1 The transport bar

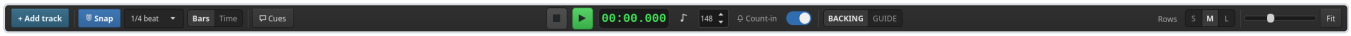


Figure 20 — *The transport bar: edit controls on the left, transport in the middle, view controls on the right.*

It sits directly above the tracks, and moves to the bottom edge when the stage is undocked.

6.1.1 Left — edit controls

Control	What it does
+ Add track	Patches a new fixture. Opens the stage designer and the fixture browser together — a lane *is* a fixture
Snap	Snapping on/off. **On by default**
1 beat / 1/4 beat / 1/8 beat	How fine the grid is. `1/4 beat` by default
Bars | Time	What the ruler counts. Bars by default
Cues	Opens the voice-cue editor — 6.1.4

6.1.2 Centre — transport

Stop, play/pause, and the position read-out (`00:00.000`).

The ♪ glyph beside the BPM is decorative — **the metronome is muted from the CLICK row** in the header column, not here.

BPM accepts 40–300. Anything outside that range is refused with a dialog naming the limits. The timeline can also detect a tempo from the audio.

6.1.3 Count-in

The **Count-in** switch is **per song**. On, the song begins with a counted-in bar or two; off, it segues straight out of the previous song. How many bars — and whether the last one is spoken aloud — is set once for the

machine in **Settings ▸ Audio** (9.3).

A count-in is *negative timeline time*: playback starts before zero and the audio, MIDI, lights and metronome all begin together at t = 0. The position read-out shows `COUNT-IN −0X.X` while it runs. Pausing during a count-in resets to zero; seeking cancels it.

6.1.4 Voice cues

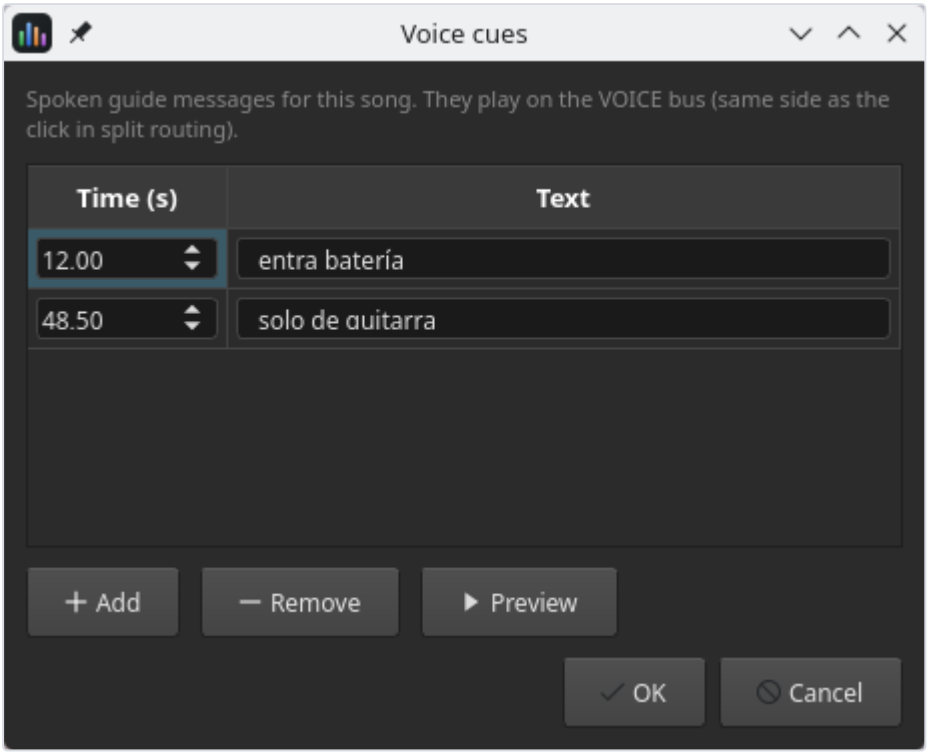


Figure 21 — *Voice cues: spoken guide messages for this song, on the voice bus.*

Cues opens a table of spoken messages for the current song — a time in seconds and some text. They play on the **VOICE** bus, which in split routing is the same side as the click, so they reach the band and not the room. **Preview** renders the selected cue and plays it.

Cues appear on the ruler as small violet markers.

Voice cues need a TTS voice installed (**Settings ▸ Audio**). Without one the dialog warns *"TTS voice not ready (Settings ▸ AUDIO) — cues stay silent."* and nothing is spoken.

6.1.5 BACKING | GUIDE

A song may carry a *guide* track — a full mix used as a design reference. This switch chooses which of the two you are monitoring and which waveform is drawn.

> The monitor is **session-only**. It is never saved, always starts on BACKING, and **never affects what a live surface plays**. MIDI, DMX, the metronome and the cues are driven by the timeline and are identical under either.

The **GUIDE** button stays disabled until the current song actually has a guide file.

6.1.6 Right — view controls

Control	What it does
Rows S | M | L	**S collapses every lane** (it is not a height). M = 50 px, L = 80 px, and both resize the timeline section to fit
Zoom slider	20 % – 400 %. Mirrors Ctrl+wheel and **Fit**
Fit	Zooms so the whole song fits the width

6.2 What a clip is

This is the one idea the whole chapter rests on.

> A clip carries a MIDI **note**, a MIDI **channel** and a **velocity**. The **note is a verb** — set a channel, load a scene, run a chase, black out. The **channel selects which fixture** the verb acts on. The **velocity is that verb's parameter**.

So a clip is always edited *against the fixture whose lane it lives on*, never against a general table of notes. The same note number means different things on different fixtures, and the app resolves it from the patch every time.

Verbs are grouped by number:

Notes	Verb
1 – 23	Set channel *n* of this fixture (DMX value = velocity × 2)
24 – 35	Load scene 0–11 of the fixture's current bank (velocity = blend time)
36 – 47	Select bank 0–11
48 / 49 / 50	Take manual control / manual / black out this fixture
51	Black out everything (a toggle)
52 – 59	Timing, palette and scene-step verbs
60	**START** — initialise the palette
61 – 69	Song parts (INTRO, VERSE, CHORUS…)
70	Run a chase
72 – 75	Colour macros

Only a fixture's first 23 DMX slots can be driven from the timeline, because notes 24 and up are already scene verbs. Channels past that are still patched, still rendered, and still reachable from the fixture's stored scenes — they just cannot be authored clip by clip.

6.3 Anatomy

The header column

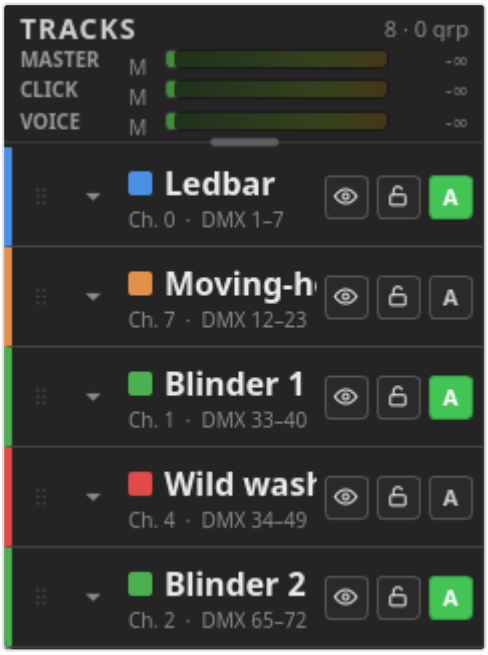


Figure 22 — *The header column: the three output buses over one header per fixture lane.*

At the top, **TRACKS** with a live count, then the three output buses. Each is a label, an **M** mute button, a level slider and a decibel read-out (`0dB` at full, `-∞` at zero):

- **MASTER** — the backing track
- **CLICK** — the metronome
- **VOICE** — the spoken count-in and voice cues

Below them, one header per fixture:

Element	What it does
grip	Reordering — *not implemented yet*, and permanently disabled
chevron	Collapses the lane to a single row. View only
colour swatch	Matches this fixture's group in the clip library and its section in the DMX monitor
name / `Ch. 0 · DMX 1–7`	The fixture's name over its MIDI channel and address range
eye	Mutes this lane's MIDI output. Its clips redraw dimmed
lock	Makes the lane read-only: no moves, no resizes, no new clips, no editing
A	Follow the song-part program automatically

The lanes

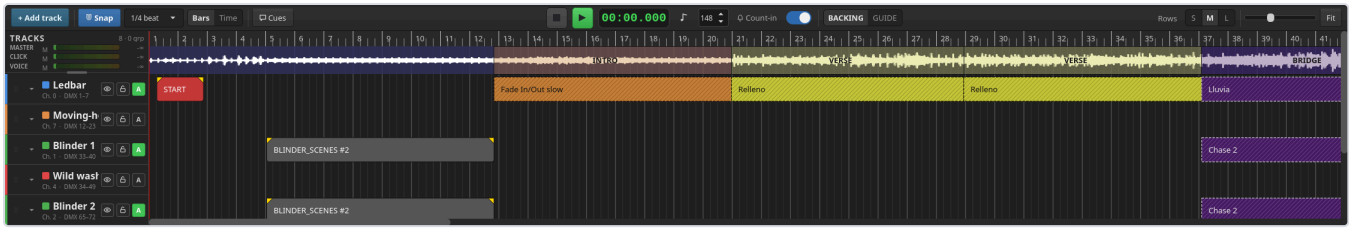


Figure 23 — *Zoomed in to a few bars: the ruler in Bars mode, the waveform with its arrangement, and clips on the fixture lanes.*

The ruler runs along the top in **Bars** or **Time**. Beneath it the waveform, overlaid with the song's arrangement as translucent labelled boxes (INTRO, VERSE, CHORUS…). Then the fixture lanes, ordered by DMX start address. A red playhead crosses the lot.

Two kinds of block appear on a lane, and telling them apart matters:

- **Clips** — solid, rounded, coloured by the verb. These are yours: click, drag, resize, edit, delete.
- **Auto blocks** — square-cornered, diagonally hatched, dashed border. These appear only on lanes whose **A** button is on. They are a *preview of the automatic song-part program*, not editable and not saved.

A clip's colour comes from its verb; a scene group's is mixed from its channel values, so a warm look is a warm block. Muted lanes draw everything at about a third opacity.

BLACKOUTs are the gaps

Between clips, a lane is not empty — it holds invisible **BLACKOUT** markers. They are never drawn and cannot be clicked. They exist so a clip's extent is explicit: a gap is a decision, not an absence.

This has one consequence you will meet within the first hour:

> `Delete Event` **does not delete a clip. It replaces it with a blackout.** That is why neighbouring clips do not expand to fill the hole. To actually remove the marker afterwards, right-click it and choose **Remove blackout**.

6.4 Undo and redo

Ctrl + Z and **Ctrl + Shift + Z**. Every committed edit is undoable, and multi-step edits collapse into a single step.

There is no undo button, no menu entry, and nothing greys out when the history is empty — pressing Ctrl+Z with nothing to undo simply does nothing.

6.5 Selecting

Action	Result
Click a clip	Selects just that one
Ctrl + click a clip	Adds it to / removes it from the selection
Drag on empty lane space	Rubber-band marquee; everything it touches is selected
Ctrl + drag on empty space	Same, but keeps the existing selection
Click empty space (no drag)	**Seeks the playhead** to that point

Selected clips get a warm gold border.

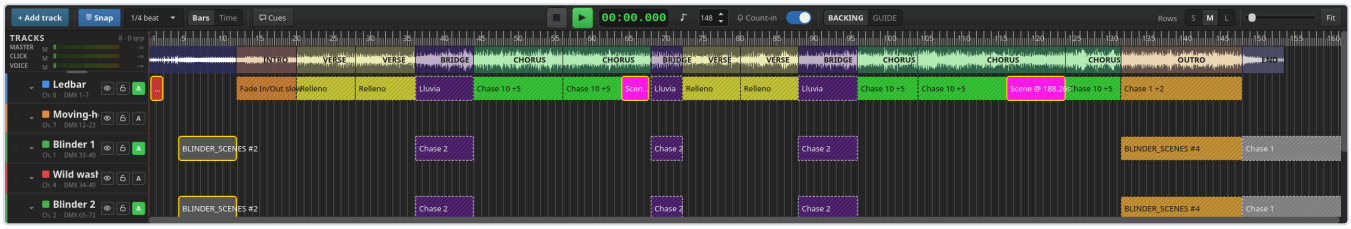


Figure 24 — *Five clips selected at once: in multi-selection the whole clip body becomes a move handle.*

With two or more clips selected the timeline enters *multi mode*: fade handles disappear, edge-resizing is switched off, the whole body of every clip becomes a move handle, and dragging any one of them slides the entire selection as a rigid block.

Selection survives redraws — it is tracked by clip, not by screen position.

6.6 Moving, resizing and fading

Grab a clip in the middle to **move** it. Grab within about 8 pixels of either edge to **resize** from that side; the cursor changes to tell you which you are about to get. On a clip too narrow to have a middle, the left half resizes left and the right half resizes right.

- **Nothing is written until you let go.** Everything during the drag is a preview.
- **Clips never overlap.** A move is clamped by its neighbours, and can hop over one when there is a big enough slot on the far side.
- **Snapping** pulls the edge you are dragging to nearby clip edges and to the finest visible grid line — bars and their subdivisions in Bars mode, seconds in Time mode. Turn it off with **Snap** on the toolbar.

Fades are the two small gold triangles on a clip's top edge: the left one is where the fade-in ends, the right one where the fade-out begins. Drag them along the clip. There are no cross-fades — each handle treats the other as a wall — and shortening a clip shrinks any fade that would have fallen off it. Fade handles are not offered on blackouts, on locked lanes, or while several clips are selected.

6.7 Adding clips

Three ways, all equivalent:

1. **Drag from the clip library** (chapter 8). The drop is validated live: over a lane where the verb has no meaning, over a locked lane, or over a fixture whose type does not match the scene you are dragging, the cursor shows *rejected*. **Song parts are the exception — they drop onto the waveform band**, not onto a fixture lane, because they belong to the song rather than to a light.
2. **Right-click a lane** and choose **Add event at this position**.
3. **Ctrl + V** pastes a copied selection at the playhead.

6.8 Context menus

Right-clicking a clip gives, in order: **Edit Properties…** (omitted on a blackout), **Add event at this position**, and **Delete Event** — or **Remove blackout** on a blackout. With several clips selected, the menu collapses to a single **Delete N events**.

Right-clicking empty lane space offers only **Add event at this position**.

Right-clicking a song-section box offers **Edit "…** and **Delete "**.

Hovering a clip shows its part name, note number, time, bar and beat, and velocity.

6.9 The clip editor

Double-click a clip, or use **Edit Properties…**. Which editor opens depends on what the clip is.

A single verb

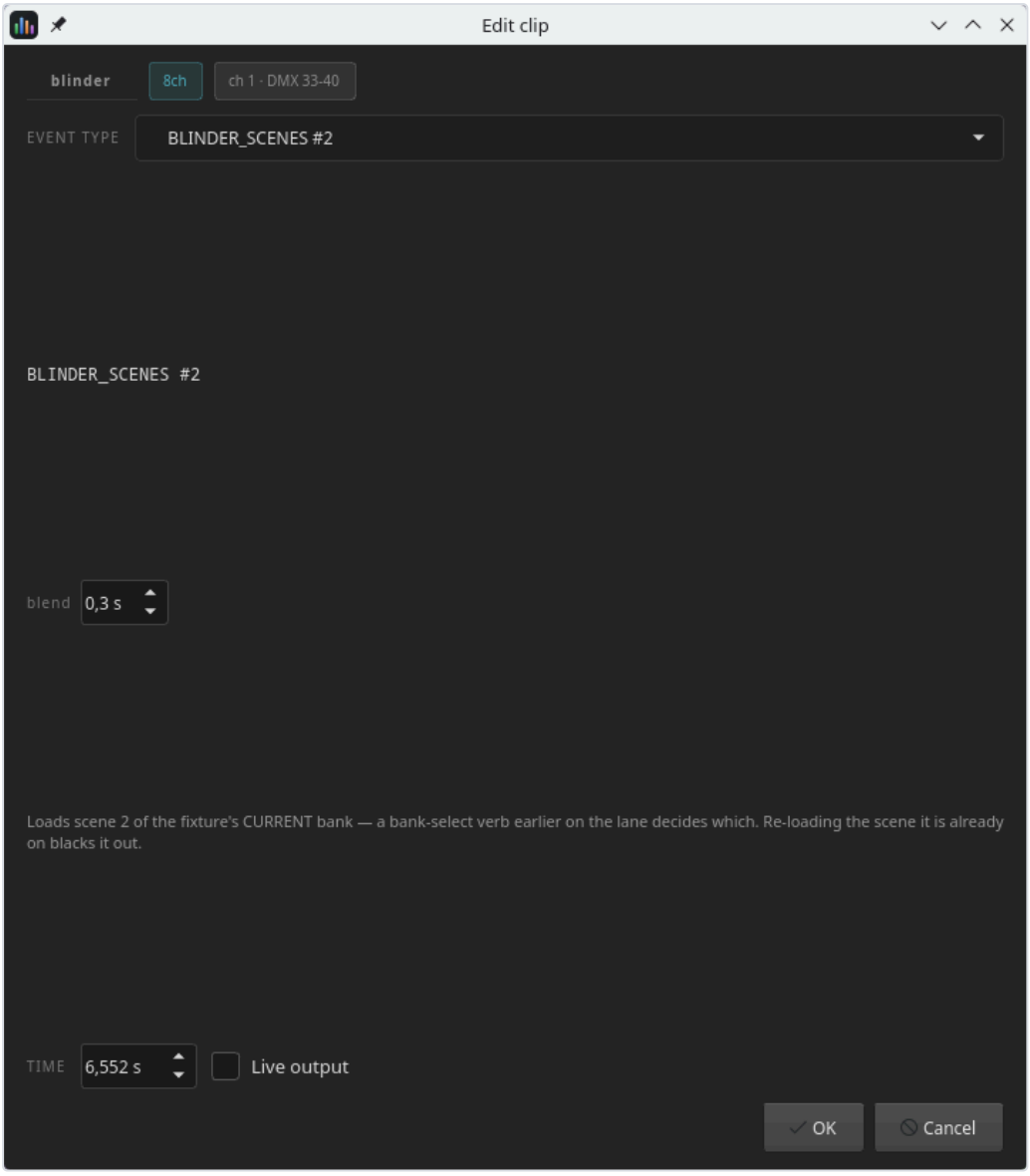


Figure 25 — *Editing one clip: the verb picker, that verb's own parameter control, and the time field.*

The header names the fixture, its mode and its placement (`ch 1 · DMX 33-40`).

EVENT TYPE lists every verb that is meaningful *on this fixture* — the picker is gated by exactly the rule drag-and-drop uses, so the dialog can never author a clip a drop would have refused. Below it sits **that verb's own control**, because a chase index and a blend time are not the same thing:

Verb	Control
Set a channel	A console fader, coloured by channel family, captioned live with what the capability under that value means (`200 of 255 — ColorIntensity` , `· 4.2 Hz` , `· 45°`)
Load a scene	The scene it targets, plus a **blend** time
Bank + scene in one	A **bank** and a **scene** drop-down
Run a chase	The chase, by name
Blend / colour / chase timing	A seconds spin, 0.0 – 12.7
Blackout, take manual, …	No parameter — just a line explaining what it does

> **Retyping a clip resets its velocity.** A channel-at-200 clip has velocity 100; carried across to "run a chase", that would select chase number 100. The editor deliberately starts the new verb from its own default.

TIME re-times the clip. It goes through the same search as a drag, so it can refuse when there is no room — the clip then simply stays where it was.

A scene group

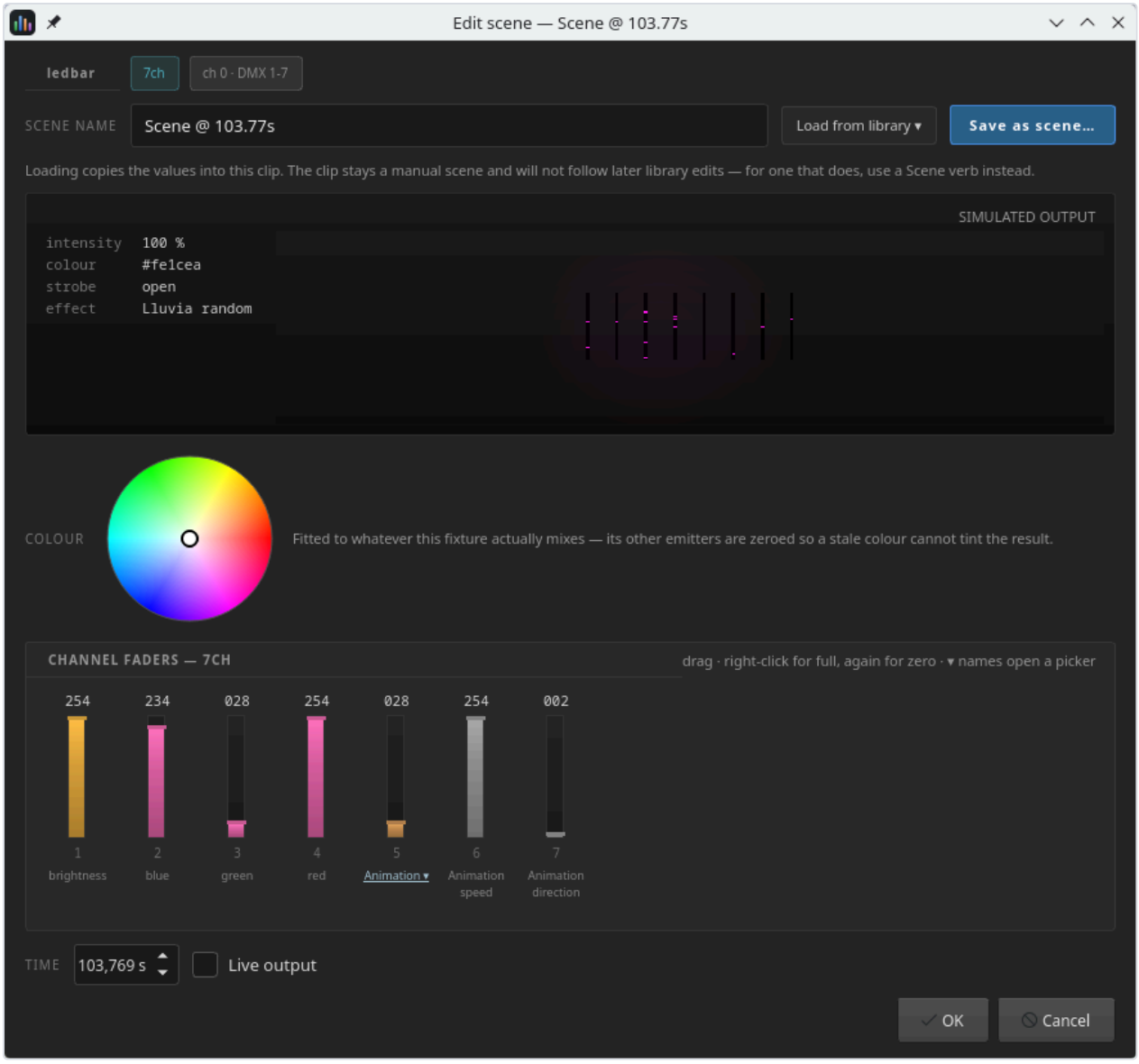


Figure 26 — *Editing a scene: the console, the colour wheel and one fader per channel.*

A *scene clip* is seven simultaneous events sharing an identity, drawn as one block. Editing it opens the full console: the scene's name, **Load from library ▾**, **Save as scene…**, the **SIMULATED OUTPUT** card, a **COLOUR** wheel (offered only when the fixture actually mixes a colour) and the fader strip.

Load from library copies a stored scene's values into this clip. The clip stays a *manual* scene and will not follow later library edits — if you want one that does, use a Scene verb instead.

The fader strip works like the one in the fixture browser: press anywhere to jump, drag to follow, right-click for full and again for zero, ▾ on a channel name to pick from that channel's own options.

Two behaviours are specific to scene editing:

- **Channels the scene does not carry are drawn dimmed.** Move one and it joins the scene. This is not cosmetic: at runtime an absent channel keeps whatever the previous frame left in it, while a present channel at its default *forces* that value. Submitting every fader would silently widen the scene.
- **Channels past DMX slot 23 are greyed out**, for the reason in 6.2.

> **"Save as scene…" writes to the fixture library immediately.** It is not part of OK, and Cancel will not undo it — it edits a fixture *type*, which every show using that type shares. If the name already exists, the app offers to overwrite in place, and recommends it: a saved song binds a scene by position, so adding a second scene with the same name shifts everything after it.

Live output

The **Live output** checkbox sends what you are dialling to the rig while you edit.

- It is **never on by default and never remembered** — this dialog gets opened in front of an audience, and a fader drag would be visible.
- It is **disabled while the transport is playing** (the sequencer would fight it for the same channels) and on a locked lane.
- Payloads lead with a *take manual* instruction, or the automatic program would overwrite the preview on the next frame.
- **Cancel re-sends what the clip held when the dialog opened.** That is a re-send, not a restore: it cannot put back what the automatic program was doing.

The START clip

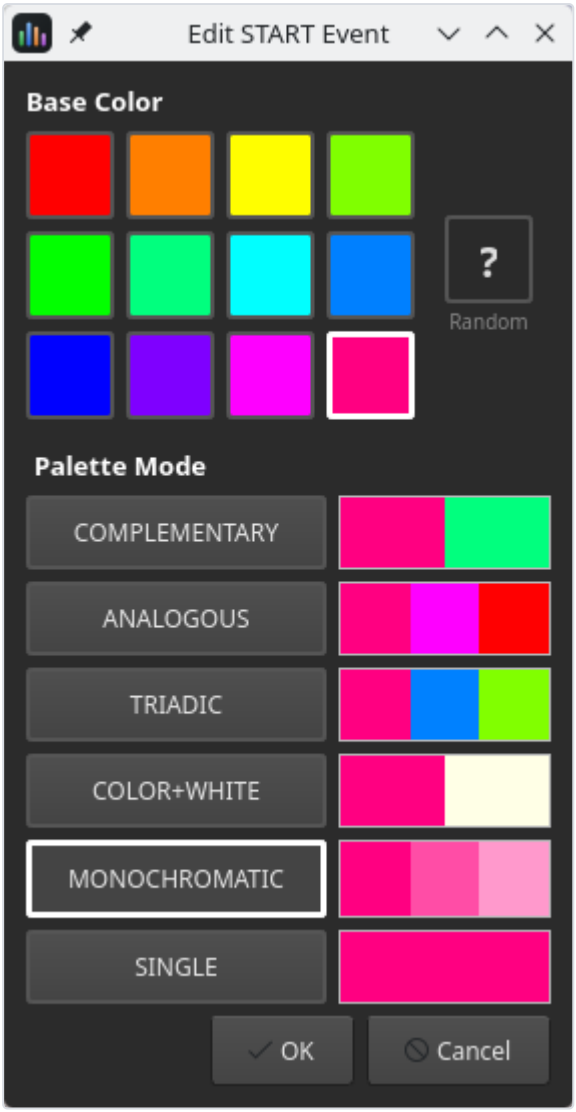


Figure 27 — *The START clip's own editor: a base hue and the palette mode built from it.*

Note 60 gets its own dialog, because its velocity packs two choices rather than one parameter: a **Base Color** from a twelve-hue grid (or **?** for random) and a **Palette Mode** — COMPLEMENTARY, ANALOGOUS, TRIADIC, COLOR+WHITE, MONOCHROMATIC or SINGLE — each previewed as a live colour band.

7. The top toolbar

Everything about the *project* lives here: which show is open, which song is selected, which files that song uses, and how far back you can roll it.

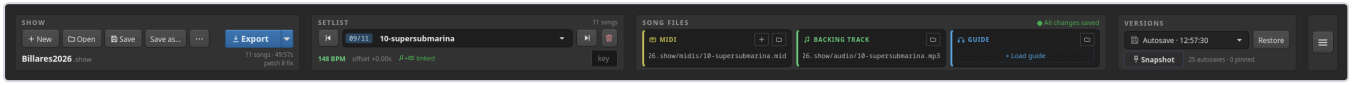


Figure 28 — *The top toolbar: SHOW, SETLIST, SONG FILES, VERSIONS and the ☰ menu.*

7.1 SHOW — the project

Button	What it does
+ New	Creates a new show project. Asks for a name, then where to put it
Open	Opens an existing `.show` folder
Save	Forces a save right now. Autosave already runs — this is for peace of mind before unplugging something
Save as…	Duplicates the show under a new name. The copy starts with no version history
…	**Import bundle…** — unpacks a portable `.showpack`
Export	See below

Underneath, the show's name and a two-line summary: `11 songs · 49:57s` and `patch 8 fix`.

A show is a folder

New shows are created under `Documents/TupperDMX/` by default and look like this:

```
Billares2026.show/
  show.json          the manifest: setlist, patch, cues
  midis/             the MIDI files this show owns
  audio/             audio that arrived inside a bundle
  fixtures/          fixture definitions that travel with this show
  versions/          the snapshot history
```

Audio is referenced where it lies, not copied — a 100 MB backing track stays in your music folder. **MIDI is copied in**: loading a `.mid` puts a copy in the show's `midis/`, and that copy is what gets edited from then on.

Export

The **Export** button is a split button, and the two halves do different things.

Clicking the button body exports a `.showpack` bundle — one portable file containing audio, MIDI, the patch and the fixture definitions for *every* song, including guide tracks. This is the "take the whole gig to

another machine" format, and it is what **Import bundle…** reads back.

The ▾ menu holds:

- **Export MIDI…** — a plain standard-MIDI copy of the *current song only*, for opening in a DAW or another lighting tool. It deliberately strips the extras that plain MIDI cannot carry (stable scene identities, scene names, per-clip fades), so it is an interchange copy, not a backup. Disabled until the song has MIDI data.
- **Export bundle** — the same thing as clicking the button body.

7.2 SETLIST — the songs

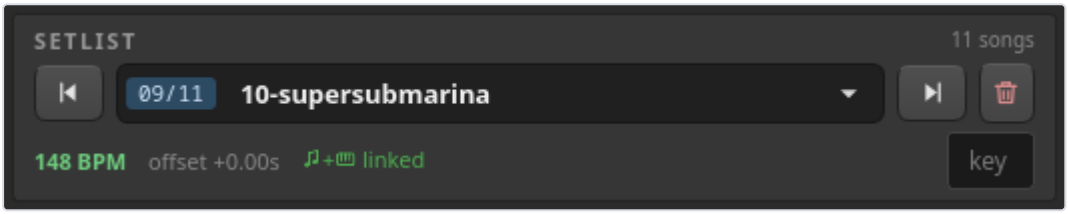


Figure 29 — *The setlist section: navigation, the song selector and the song's summary chips.*

Control	What it does
◀ / ▶	Previous / next song. The first and last disable the matching arrow
09/11	Position in the setlist
Song selector	**Pick from the drop-down to switch songs. Type a new title and press Enter to rename the current one**
🗑	Removes the current song from the setlist (with confirmation). The files stay on disk

The footer shows the song's tempo, its audio offset, a media chip, and a small **key** field — the musical key, which is shown on the web remote for the band.

The media chip is worth learning:

Chip	Meaning
`♪+🎹 linked` (green)	Backing track and MIDI both present
`♪ audio only`	A backing track, no lighting yet
`no audio`	MIDI only
`⚠ audio missing` (red)	The file the song points at is not there
`⚠ MIDI missing` (red)	Same, for the MIDI file

When files go missing

Moving or renaming files behind the app's back is normal, and it copes. When a song's backing track cannot be found, the app first tries to relink silently — if exactly one known-good folder (the show's own `audio/`,

the folder you last relinked from, or the folder another song lives in) contains a file of that name, it is adopted without asking. Otherwise you are offered a file picker.

7.3 SONG FILES — the current song's media

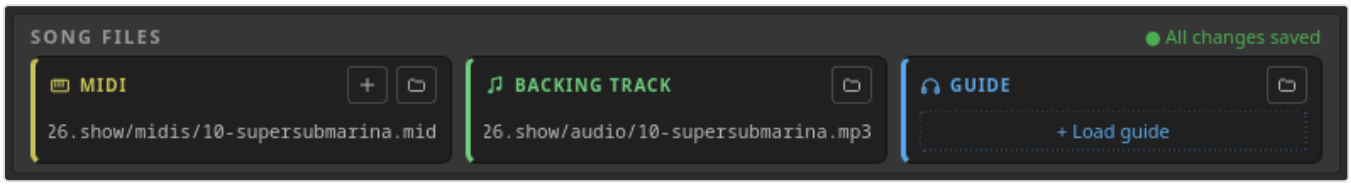


Figure 30 — *The three file cards, with the autosave chip in the header.*

Three cards, one per slot. Each shows the path in use, or a dashed **+ Load …** button when the slot is empty. The small 🗁 in a card's header loads a file into that slot.

Card	What it is
MIDI	The song's lighting. The **+** in its header starts a fresh empty track. Loading a `.mid` **copies it into the show**
BACKING TRACK	The song's identity — this is what plays live, what the web remote controls, and what a headless rack box runs
GUIDE	Optional full mix used as a design reference. **Never played live.** The **✕** in its header clears it (the file itself is untouched)

The autosave chip

Top-right of this section, and it has exactly three states:

Chip	Meaning
● **All changes saved** (green)	Everything is on disk
● **Editing · autosave in 3s** (amber)	An edit is pending; it will be written shortly
⚠ **Save error — check logs** (red)	Something went wrong writing. Check the log file

You do not need to save. The show is written a few seconds after you stop editing, and always on song switch, on close, and when you press **Save**.

7.4 VERSIONS — history

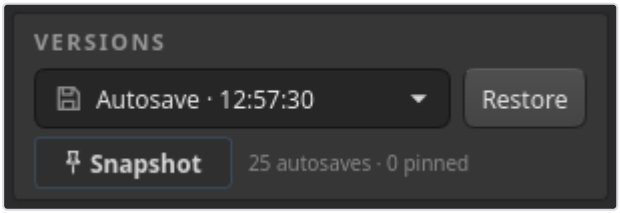


Figure 31 — *The versions section: pick a snapshot, restore it, or pin a new one.*

Every save also leaves a point you can come back to.

Control	What it does
Snapshot drop-down	The history, newest first. Autosaves show a save icon and read `Autosave · 12:57:30`; pinned snapshots show an amber pin and their name
Restore	Rolls the show back to the selected snapshot
Snapshot	Takes a named snapshot that is **pinned** — never pruned

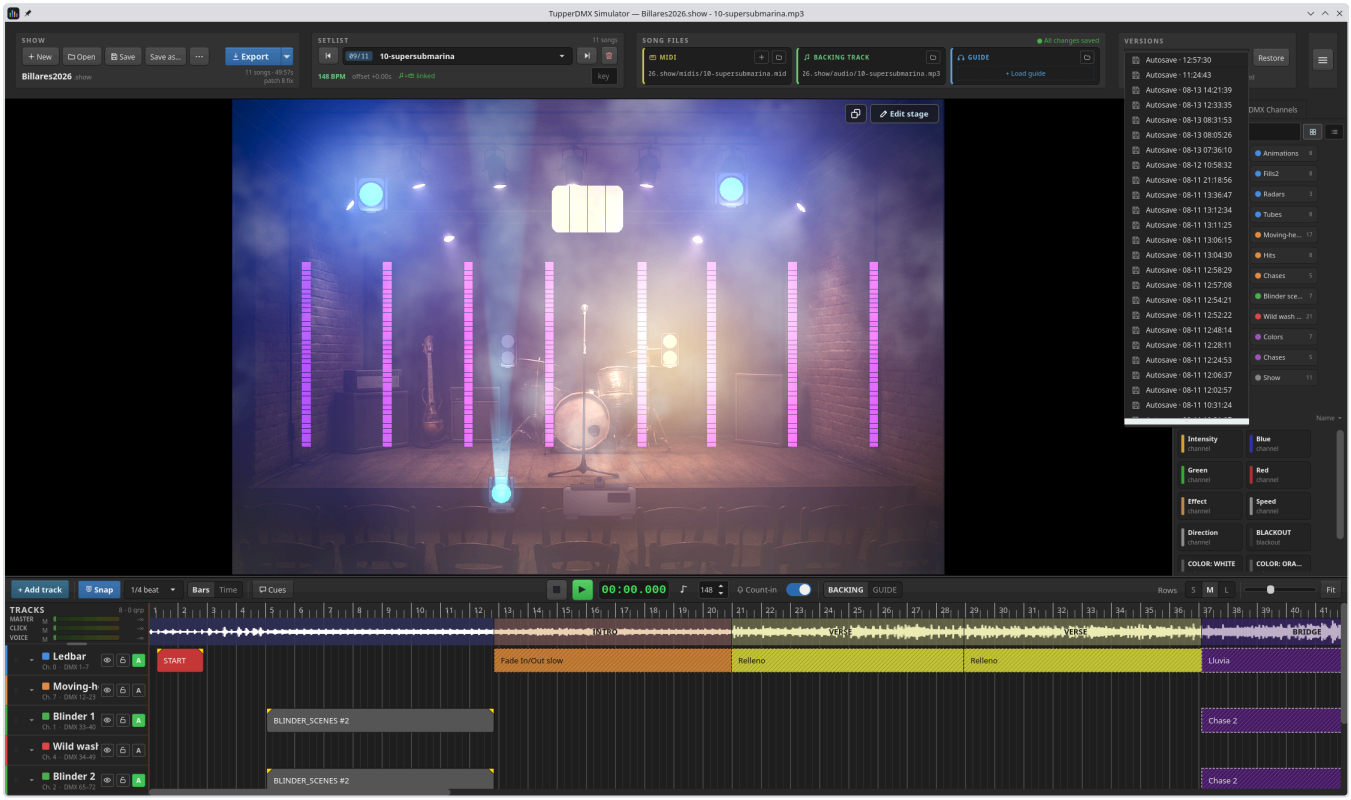


Figure 32 — *The version history dropped down: pinned snapshots by name, autosaves by time, newest first.*

Times show as `HH:MM:SS` for today and `MM-DD HH:MM:SS` for older points.

Automatic snapshots are taken when a show is opened and at most once every five minutes thereafter; older ones are eventually pruned. Anything you name with **Snapshot** is kept forever.

> **Restoring is itself undoable.** Before rolling back, the app snapshots the current state — so if the version you restored was the wrong one, the state you just left is sitting at the top of the list.

7.5 What is saved where

Lives in the show folder	Lives in this machine's preferences
The setlist and each song's settings	Audio device and routing
Which song is current	I/O routing and Art-Net target
The fixture patch	Web-remote on/off and port
The web-remote cues	Count-in bars, TTS voice
MIDI files, bundled audio, show-local fixtures	Stage haze, GPU renderer, fog quality
The version history	Window and track sizes, recent shows

The split is deliberate: a show should carry the gig, and preferences should carry the box. Copying a show to another machine never drags an audio device or a graphics setting along with it.

8. The clip library and the DMX monitor

The right sidebar has two tabs. **Library** is where cues come from; **DMX Channels** is where you check what actually went out. The chevron at the top of the stage slides the whole sidebar away when you want the room.

8.1 The Library tab

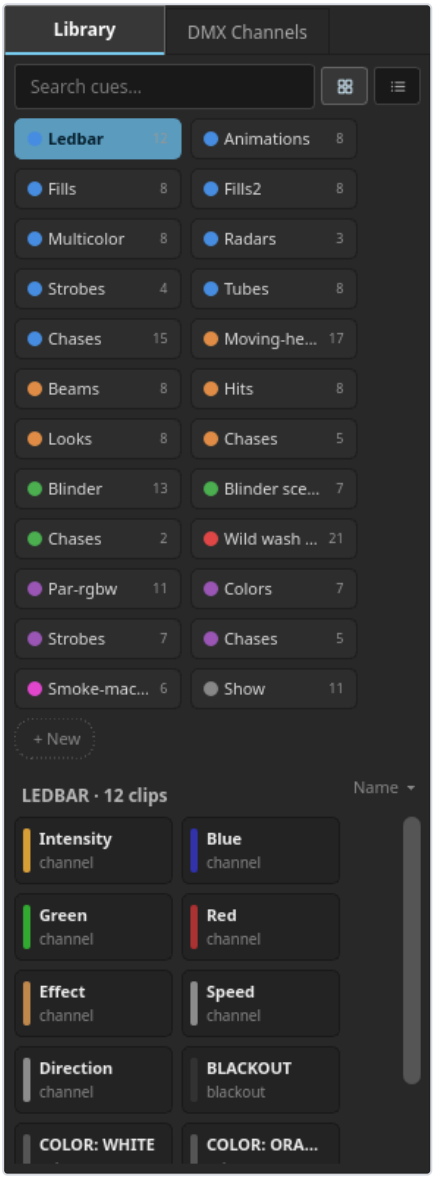


Figure 33 — *The clip library: group pills built from the patch, and the clips of the selected group.*

Everything here is built from **your patch**. Patch a fixture and its cues appear; remove it and they go.

Groups

The coloured pills at the top are the groups, and each fixture contributes several:

- **one group named after the fixture** — its individual channel controls

- **one group per scene bank** the fixture type declares (`Fills`, `Multicolor`, `Strobes`, `Beams`, `Looks`…)
- **a `Chases` group** for that fixture's chases
- and, once for the whole show, a grey `Show` group holding the global verbs: START, the song parts, and blackout-all

A pill's colour dot matches that fixture's timeline lane and its section in the DMX monitor, so a colour means the same fixture everywhere.

Cards

Each card is one cue: a title, a subtitle naming its kind (`channel`, `scene`, `chase`, `color`, `blackout`, `cue`) and a coloured accent bar. A small ✱ on a scene means its colour comes from the palette rather than from fixed values, so it follows whatever the START clip set up.

Two things you can do with a card:

Gesture	Result
Click	Auditions the cue live through whatever output is routed. **The timeline is not touched**
Drag onto a lane	Inserts it as a clip at the drop point

Auditioning goes through the same path as everything else, so it respects your I/O routing — with the factory settings that means the internal controller and the virtual stage.

Search

The search box (`Search cues…`) spans **every** group at once, not just the selected one, and the header switches to `SEARCH · N clips` while it is active. It is the fastest way to find a scene when you cannot remember which bank it went into.

> **Not yet implemented:** the grid/list view buttons, the `Name ▾` sort label and the `+ New` tile at the end of the pill row are placeholders. They are drawn but do nothing.

If nothing is patched, the panel says so: *"No fixtures patched. Patch a fixture (Design stage) or edit the fixture defs."*

8.2 The DMX Channels tab



Figure 34 — *The DMX monitor in USED view: one section per patched fixture, values live.*

This is the universe as it is right now. Values update live with the show.

Two views, chosen by the segmented buttons at the top:

USED · N groups the channels by fixture — a section per patched fixture with its name, its colour swatch and its address range (`001-007`), and a cell per channel. Each cell shows the channel number and its current value, with a coloured fill rising in proportion.

FULL 512 shows the whole universe in order, with unpatched channels dark and dimmed. This is the view for "is anything at all being sent on 214?"

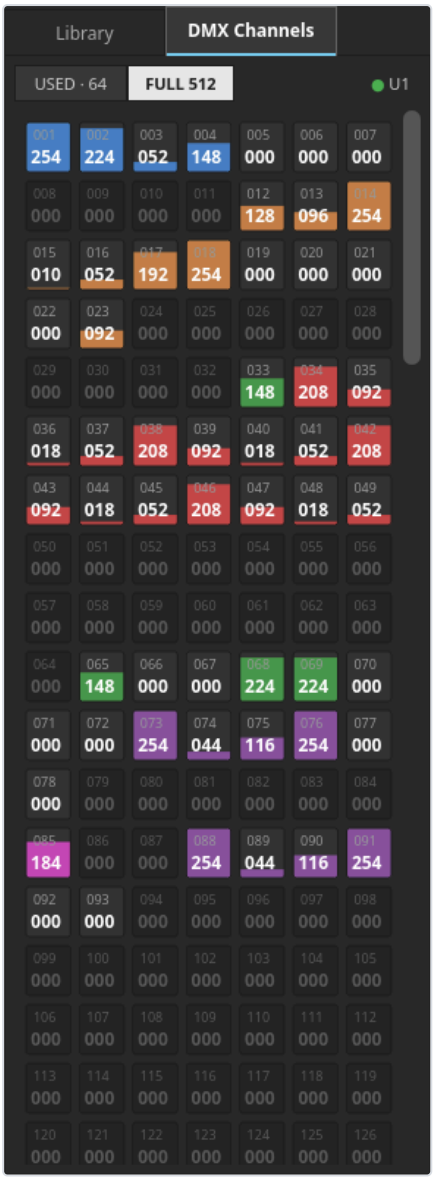


Figure 35 — *FULL 512: the whole universe, patched channels coloured, everything else dark.*

The ● **U1** chip on the right names the universe being shown.

Editing channels by hand

While the internal controller is running, the monitor is also a crude console:

Gesture	Result
Drag a cell vertically	Fades that channel. The cursor becomes a vertical fader
Right-click a cell	Toggles it between full and zero

> These edits are **transient**. The next scene, chase or MIDI event overwrites them — which is exactly what you want from a "poke it and see" tool, and exactly wrong as a way to build a look. Build looks in the fixture Scenes tab or in a scene clip.

Hand editing is only available when there is an internal controller to write to. With the stage fed from an external Art-Net or USB source, the monitor is read-only.

9. Settings

☰ ▸ **Settings…** Five pages, chosen from the sidebar. Everything here describes **this machine**, never the show — so a laptop and the rack box can be routed completely differently while running the same project.

Changes take effect on **OK**. Two of them explicitly wait for the next launch, and say so.

9.1 Input / Output

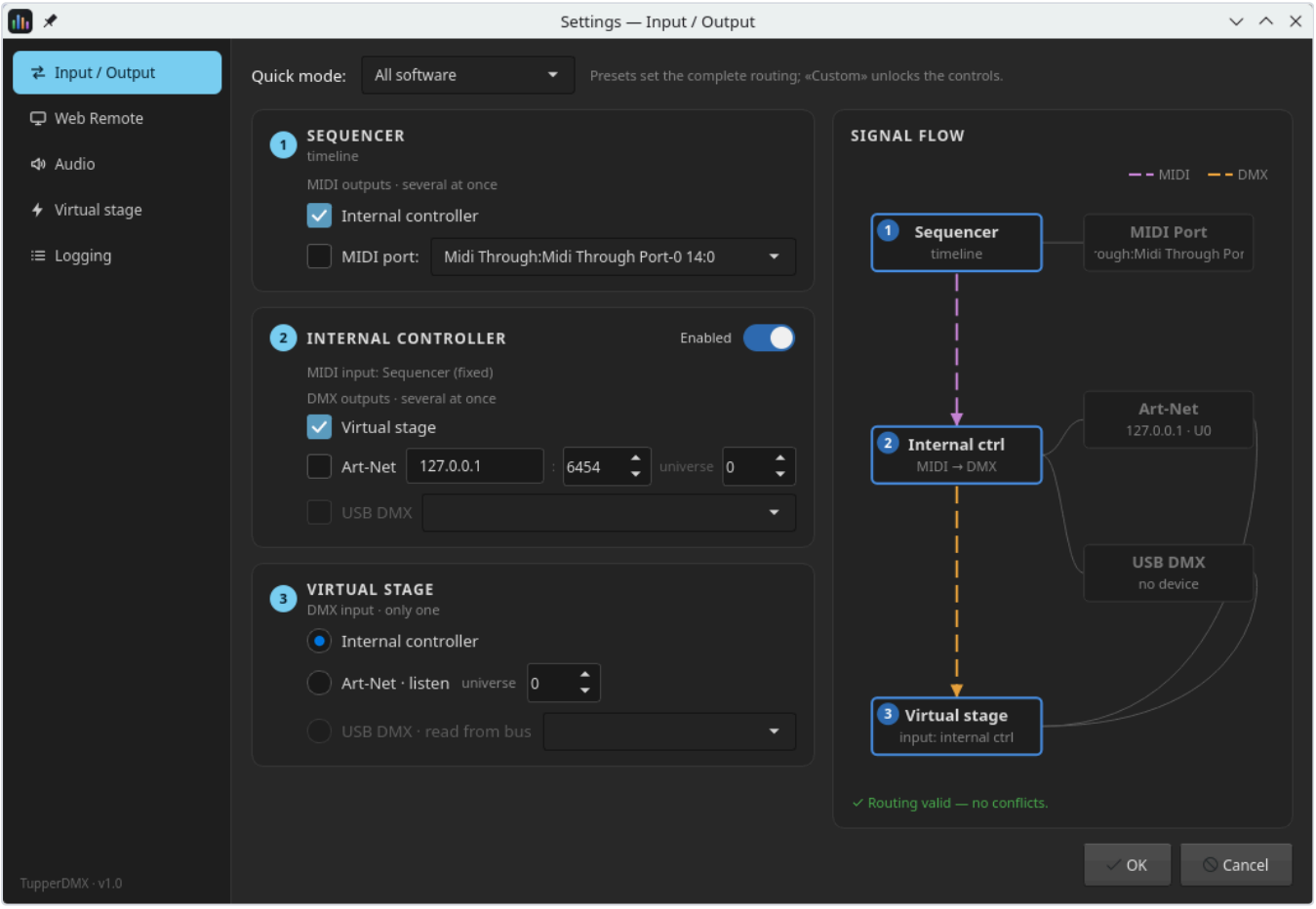


Figure 36 — *The I/O page: three numbered cards on the left, the live signal flow on the right.*

This is the most important page in the application, and the one worth reading before anything stops working.

There is **no localhost magic and no toolbar selector**: all routing is explicit here.

Quick mode

The preset drop-down sets a complete routing in one go. Picking **Custom** unlocks the individual controls.

Preset	What it is for
All software	Designing with no hardware at all. The internal controller runs and feeds the virtual stage in-process. **This is the default**
Software + real lights	The same, plus Art-Net out to a real rig
External rig monitor	Nothing is generated here; the stage listens to Art-Net and visualises somebody else's desk
Sequencer for HW rig	The timeline drives an external hardware controller over MIDI, while the internal one still feeds the stage

The three cards

They are numbered because the signal really does flow through them in order.

① **SEQUENCER** — the timeline's MIDI output. It can feed several destinations at once: the **Internal controller** and/or an external **MIDI port**. (The port drop-down disables itself when the system reports no MIDI outputs.)

② **INTERNAL CONTROLLER** — the built-in lighting desk, with its own on/off switch in the card header. Its MIDI input is fixed (the sequencer); its DMX output can go to several places at once: the **Virtual stage**, **Art-Net** (with IP, port and universe) and **USB DMX** (with a device).

③ **VIRTUAL STAGE** — where the stage gets its DMX *from*, and **only one source is allowed**: the internal controller, an Art-Net universe it listens to, or a USB DMX interface it reads from the bus.

The cardinalities are not arbitrary:

- **MIDI output is one-to-many, but MIDI input is one-to-one.** The controller speaks a stateful command protocol — "select bank 2", then "load scene 3" — so fanning *out* is safe while merging two senders *in* would interleave two conversations into nonsense.
- **DMX output is one-to-many**, because each frame is a complete snapshot.
- **Stage input is exactly one**, for the same reason in reverse.

Signal flow

The right-hand panel draws the routing you have configured — boxes for each stage, dashed lines coloured by MIDI or DMX, and greyed boxes for destinations that are switched off. Underneath, a validation strip reads ✓ **Routing valid — no conflicts**, or lists exactly what is wrong. OK refuses an invalid routing.

9.2 Web Remote

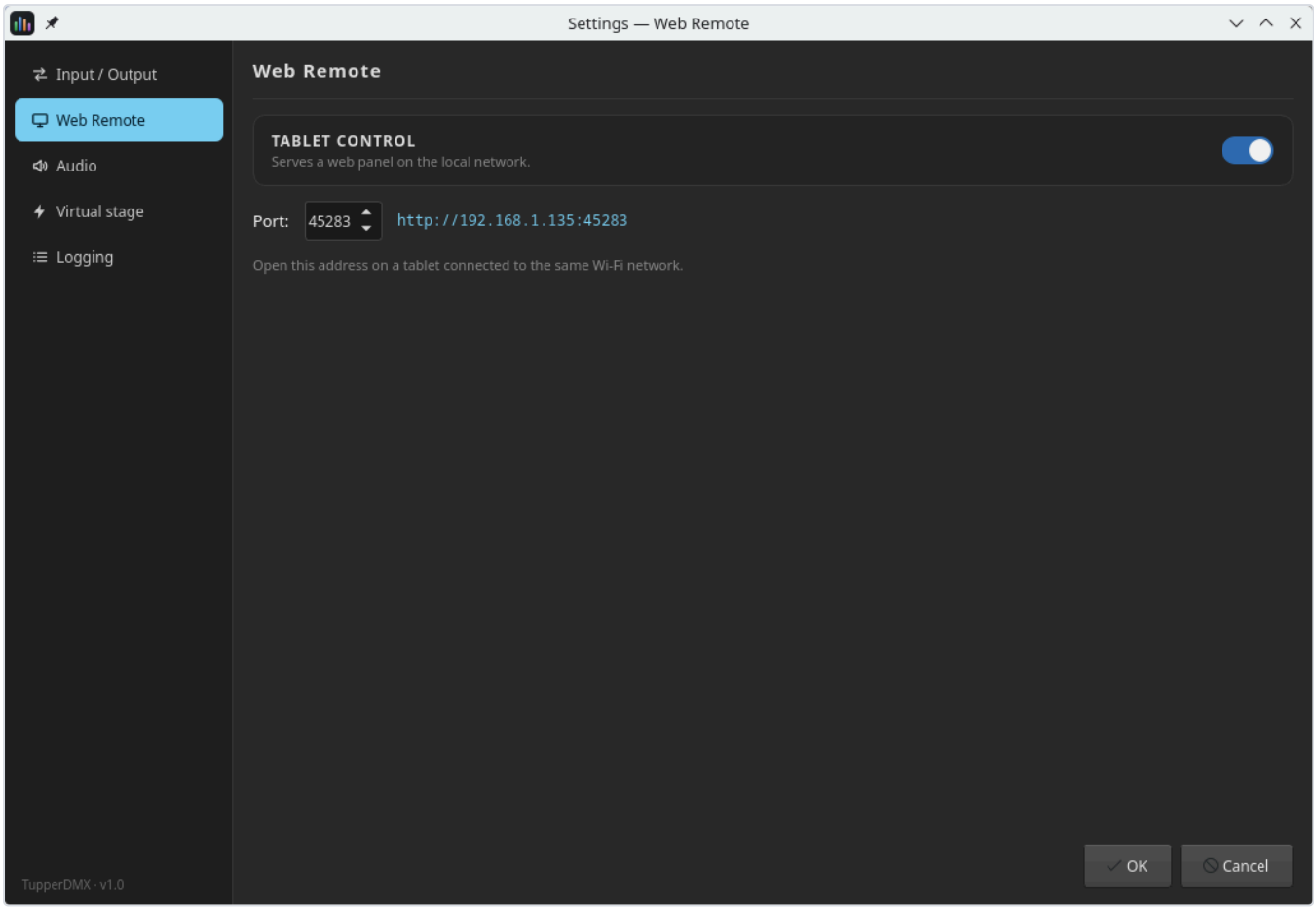


Figure 37 — *The Web Remote page: the switch, the port, and the address to type into a tablet.*

One switch, one port, and the address the server is reachable at. Open that URL on a tablet on the same network. Chapter 10 covers the panel itself.

While the remote is off, the line reads *"Disabled — no server runs."* — no socket is opened at all.

9.3 Audio

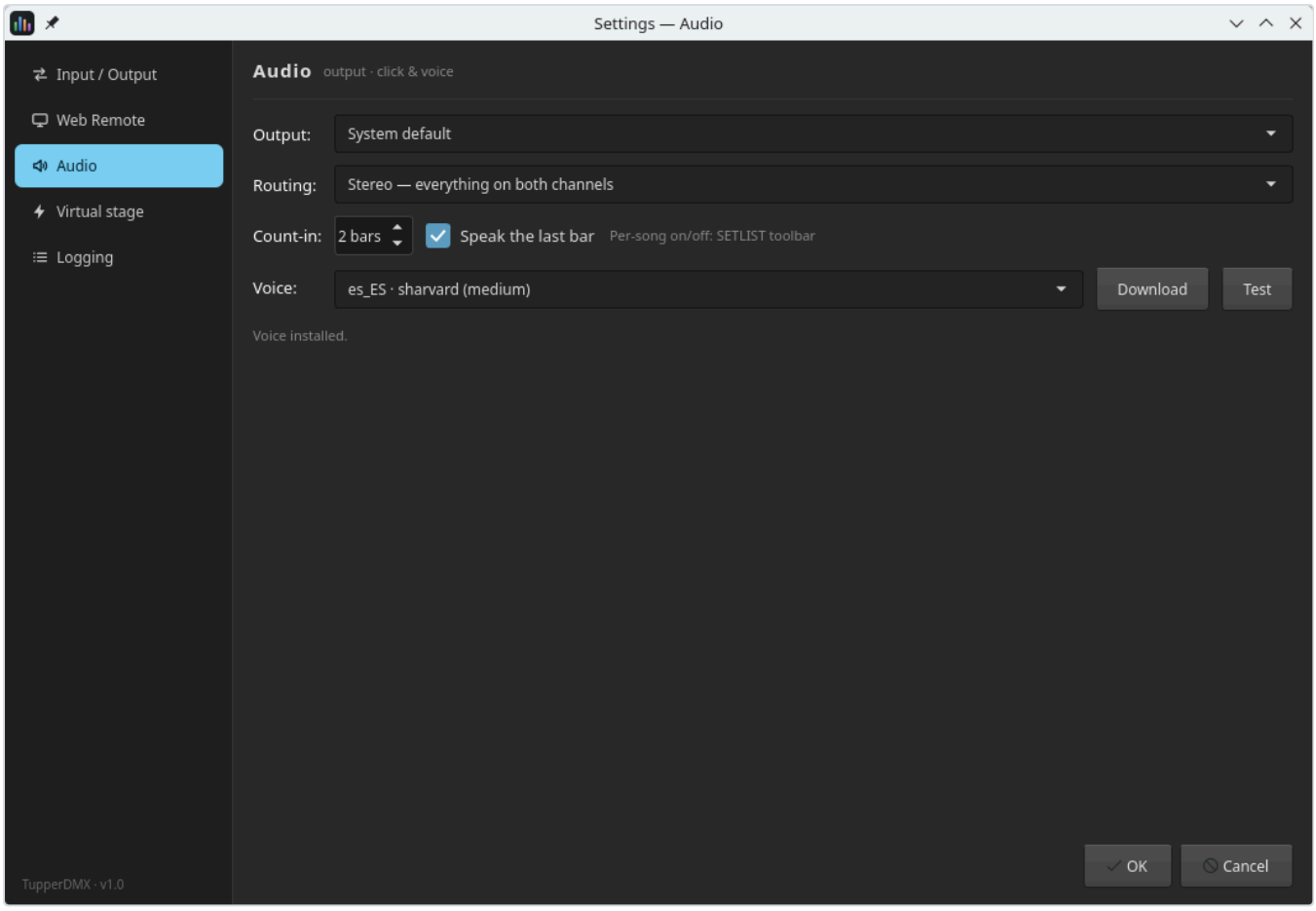


Figure 38 — *The Audio page: output device, stereo routing, count-in and the TTS voice.*

Output picks the device. A device you have configured but unplugged stays in the list marked `(not connected)`, so it is still selected when you plug it back in.

Routing decides how the three buses reach the two channels:

Mode	Left	Right
Stereo	everything	everything
Split — backing LEFT · click+voice RIGHT	backing	click + voice
Split — backing RIGHT · click+voice LEFT	click + voice	backing

The split modes are what make a single stereo output usable live: send one channel to the PA and the other to the in-ears, and the room never hears the click.

Count-in sets how many bars (1–8) and whether the last bar is **spoken aloud**. Whether a *particular song* gets a count-in at all is a per-song switch on the timeline toolbar — the hint beside the checkbox says so.

Voice picks the text-to-speech voice used for the spoken count-in and for voice cues. Voices are about 60 MB and are **downloaded on demand** — they are never bundled with the application. **Download** fetches the selected one; **Test** speaks a phrase through the voice bus. The status line below says where you stand:

"Voice installed.", *"Voice not downloaded yet (~60 MB, one time)."*, or *"TTS engine (piper) not installed — voice features disabled."*

Nothing is ever synthesised during playback — everything is rendered ahead and cached — so a missing voice degrades to silence rather than to a stutter.

9.4 Virtual stage

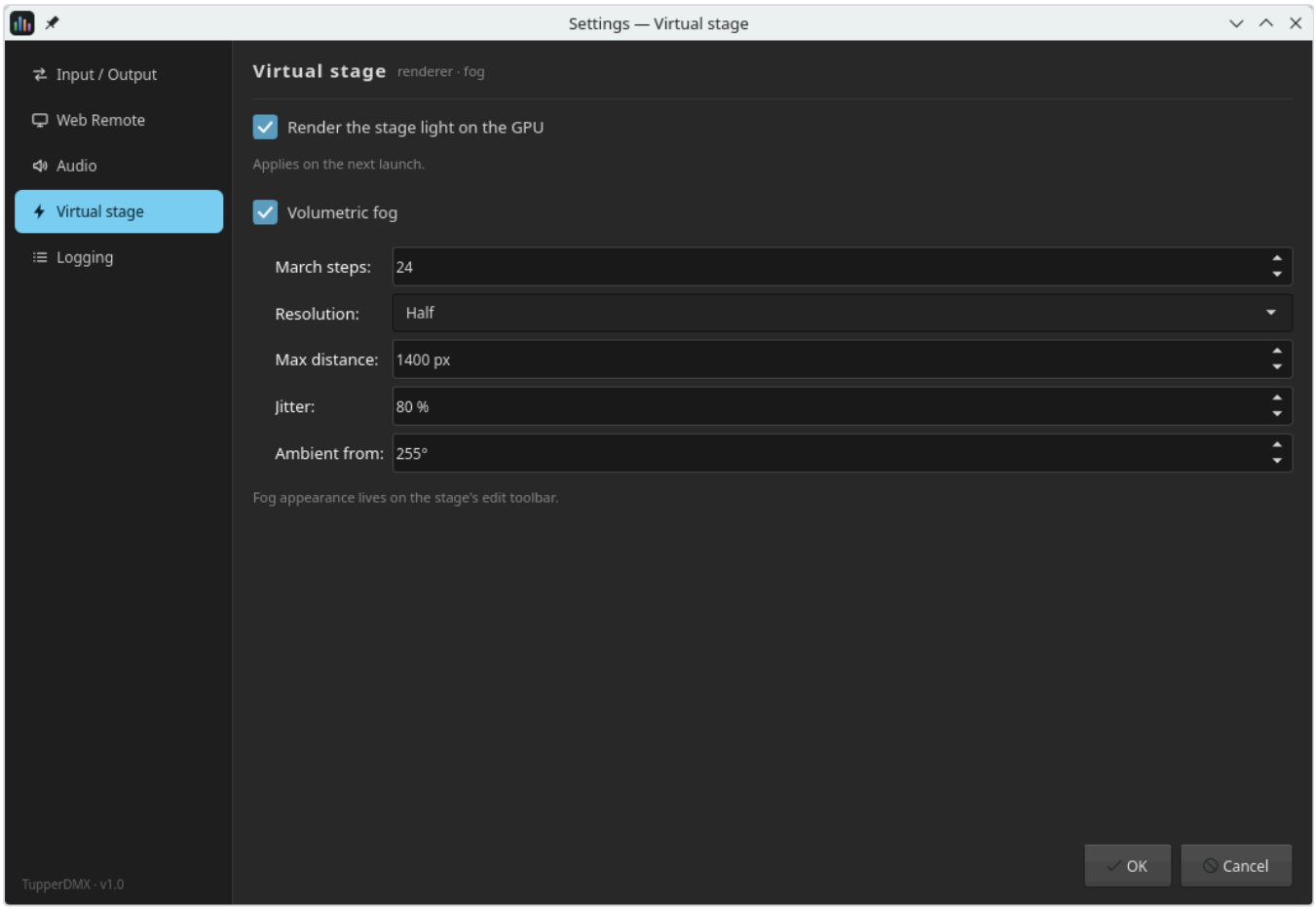


Figure 39 — *The Virtual stage page: the GPU renderer and the cost of volumetric fog.*

Render the stage light on the GPU accumulates every fixture's light into one high-dynamic-range buffer and tonemaps it with a filmic curve, so beams *add* where they cross and blow out instead of flattening to white. It needs a working OpenGL driver, and falls back to the normal renderer if there is not one. **It applies on the next launch.**

Volumetric fog needs the GPU renderer above, and takes effect immediately. Its cost controls live here because they are a property of the machine:

Setting	Notes
March steps	4–128. 24 is comfortable on integrated graphics
Resolution	Quarter (fastest) / Half / Full (slowest)
Max distance	How far a ray is marched, in pixels
Jitter	0–200 %. At 0 the banding comes back
Ambient from	The direction the ambient light comes from, in degrees

> **How the fog *looks* is not set here.** Height, falloff, scattering, texture and colour are on the stage's own edit toolbar (4.3), where you can see what you are changing. And there is no density control anywhere on this page, because density is the **HAZE** slider on that same toolbar.

9.5 Logging

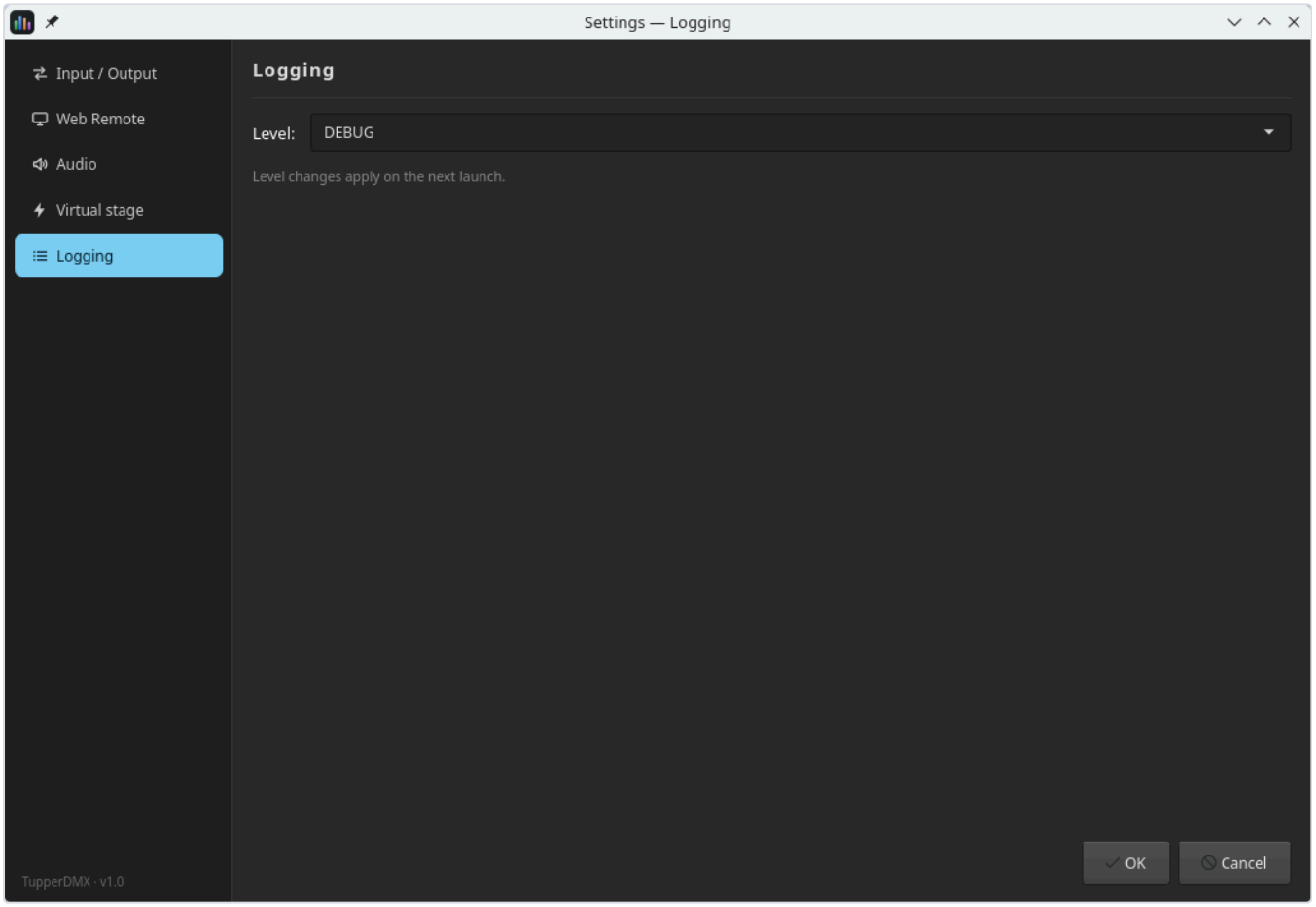


Figure 40 — *The Logging page: one setting, and it waits for the next launch.*

The log level (DEBUG, INFO, WARNING, ERROR, CRITICAL). **It applies on the next launch.** Logs are written to a dated file in the `logs/` folder next to the application — the first place to look when the autosave chip goes red.

Leave it on INFO for normal use. DEBUG is worth turning on before reproducing a problem you intend to report, and worth turning off again afterwards.

9.6 Resetting

Every setting on these five pages lives in one file:

Platform	Path
Linux	`~/.config/tupperDMX/userprefs.json`
Windows	`%APPDATA%\tupperDMX\userprefs.json`
macOS	`~/Library/Application Support/tupperDMX/userprefs.json`

Delete it to return the application to factory settings. **No show data is in that file** — your projects are separate folders, with their own history.

10. The web remote

The web remote is a page served on your local network, meant for a tablet on a mic stand. It is how the show is actually run: the laptop can be closed in a flight case behind the drum riser.

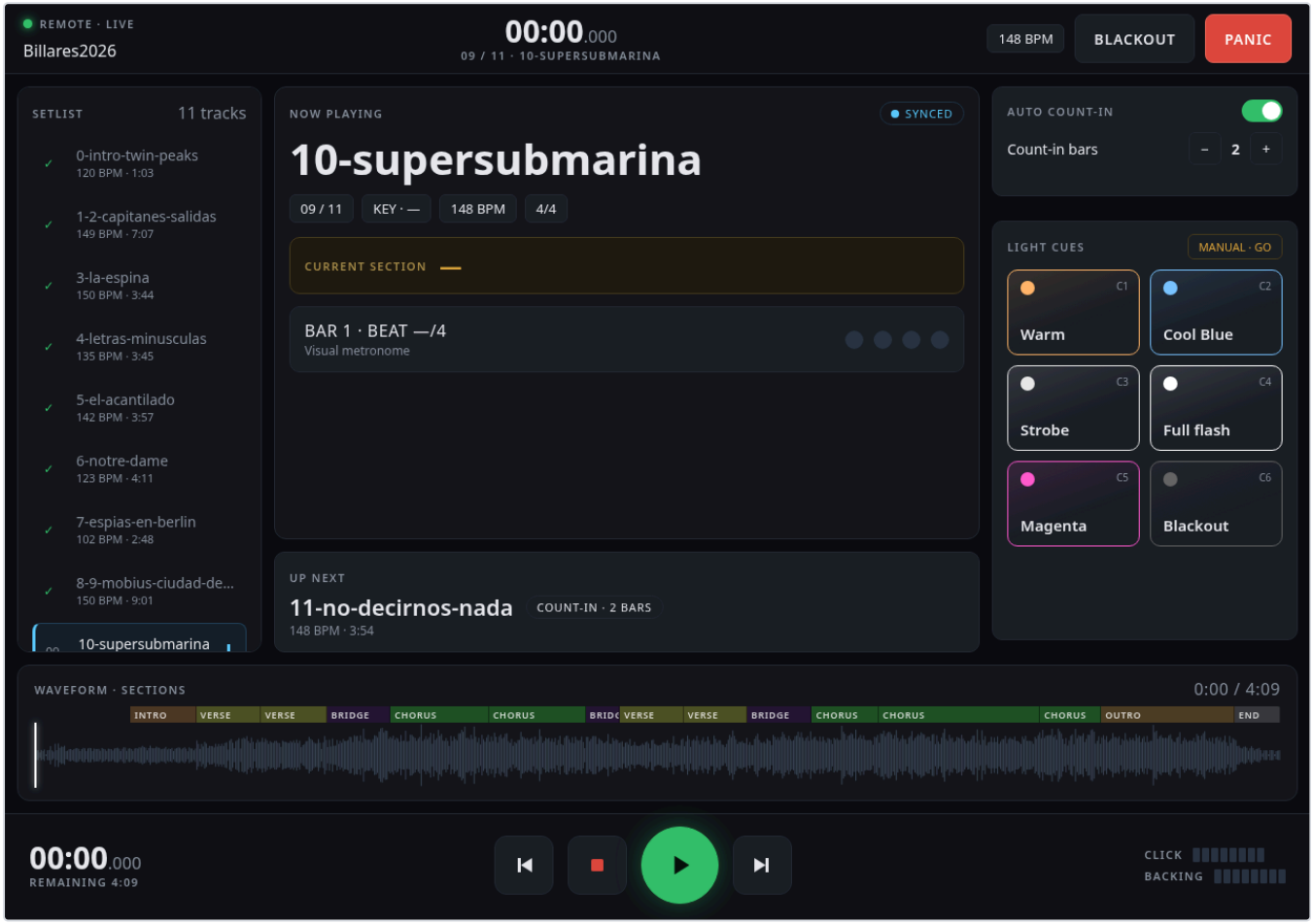


Figure 41 — *The remote on a tablet: setlist, now playing, arrangement, waveform, count-in, light cues and transport.*

10.1 Turning it on

Settings ▸ Web Remote — flip the switch, pick a port, and open the address it shows on any device on the same network. Nothing is served while the switch is off.

> **There is no password.** The remote is plain HTTP with no authentication: any device that can reach the address can run your show. That is a deliberate trade — a login screen on a tablet at the side of a stage, in the dark, thirty seconds before a set, is a worse problem than the one it solves. **Put it on a network you control.** Do not expose the port to the internet, and prefer a dedicated stage router over the venue's guest Wi-Fi.

10.2 The panel

Top bar

The show's name on the left, the running clock and `09 / 11 · SONG TITLE` in the middle, and on the right the tempo chip, **BLACKOUT** and **PANIC**.

BLACKOUT is a toggle — press it once to kill the rig, again to bring it back.

PANIC stops the show and forces blackout on. It is not the same button: panic never toggles the lights back up, so hitting it twice in a bad moment cannot re-light the stage.

SETLIST

Every song in order, with its tempo and duration. Tap one to switch to it. The current song is highlighted.

NOW PLAYING

The song title, its position in the setlist, its key, tempo and time signature, and a **sync pill** that reads `SYNCED` while the connection to the application is alive and `DISCONNECTED` when it is not. Watch that pill — it is the difference between "the show is not responding" and "the tablet fell off the Wi-Fi".

CURRENT SECTION names the part of the arrangement you are in, with a countdown to the next one, so a musician can see the chorus coming.

BAR — · BEAT —/4 is a visual metronome: four dots, lit on the beat, for when the click is not in your ears.

WAVEFORM · SECTIONS

The song's waveform with its arrangement laid out above it as coloured blocks — INTRO, VERSE, CHORUS, BRIDGE, OUTRO, END. The playhead runs across it, and **tapping the waveform seeks**.

UP NEXT

Which song follows, its tempo and length, and whether it starts with a count-in.

AUTO COUNT-IN

The current song's count-in switch and a **− 2 +** stepper for the number of bars. Both write straight back to the show — this is the same setting as the timeline toolbar's, reachable without walking back to the laptop.

LIGHT CUES

A grid of manual looks, labelled `C1` … `Cn` with a colour dot. Tapping one fires it immediately.

Cues come from the **open show**, so each project carries its own. They travel inside a `.showpack` bundle with everything else.

Transport

Along the bottom: the elapsed clock and time remaining, a seek bar, previous / stop / play-pause / next, and two level meters for **CLICK** and **BACKING**.

10.3 Everything routes normally

Cues, blackout and panic go through the same path as a click in the desktop clip library, so your **Settings ▸ Input / Output** routing applies unchanged. A cue fired from the tablet reaches the same lights, over the same interfaces, as one fired from the laptop.

10.4 Rack mode

```
uv run python main.py --headless
```

Headless mode runs the whole application without a desktop window — controller, DMX and MIDI output, playback — and force-enables the remote so there is a way in. That switch is only in memory: it never rewrites the preferences of the machine it is running on.

This is the configuration for a small box living in the rack: it boots into the show, and the tablet is the only interface anyone touches all night.

11. A show from start to finish

This chapter is one continuous worked example: an empty project on a Monday, a gig on a Saturday. Each step points at the chapter that covers it properly.

Step 1 — Create the show

SHOW ▸ + New. Name it after the gig or the tour (`Billares2026`), and choose where it lives; `Documents/TupperDMX/` is the default and is fine.

You now have a folder. Everything from here — the setlist, the rig, the cues — goes inside it. → 6.1

Step 2 — Load the first backing track

SONG FILES ▸ BACKING TRACK ▸ + Load backing, and pick the MP3.

The song appears in the setlist, named after the file. Rename it by typing over the title in the **SETLIST** selector and pressing Enter.

Repeat for the rest of the set. Songs are added in the order you load them. → 6.2

Step 3 — Get the tempo and the offset right

Set the **BPM** on the transport bar, or let the timeline detect it from the audio. Then check the grid against the waveform: if the bar lines drift off the downbeats, the tempo is close but wrong.

If the bar lines are *parallel* to the beats but shifted, the problem is the offset — **drag the waveform horizontally** to slide the audio against the grid until bar 1 lands on beat 1.

Get this right before writing any lighting. Every clip you place is positioned against this grid. → 6.1.2

Step 4 — Patch the rig

Edit stage ▸ Add fixture, once per light.

For each one: find it in the library (search by manufacturer or model), check the **Modes** tab and pick the mode your fixture is actually set to, then set the **DMX START** and the **MIDI CHANNEL** in the footer. Watch the patch bar — the block turns red if the address collides, and **OK** stays disabled until the patch is legal.

Then drag the fixture to where it stands on the real stage. Drop it high for a flown fixture, low for one on the deck; correct the guess in the Inspector's **MOUNTING** switch if it got it wrong.

Set the MIDI channel once and leave it. Every clip you write from now on is bound to its fixture by that channel. → 5, 4.5

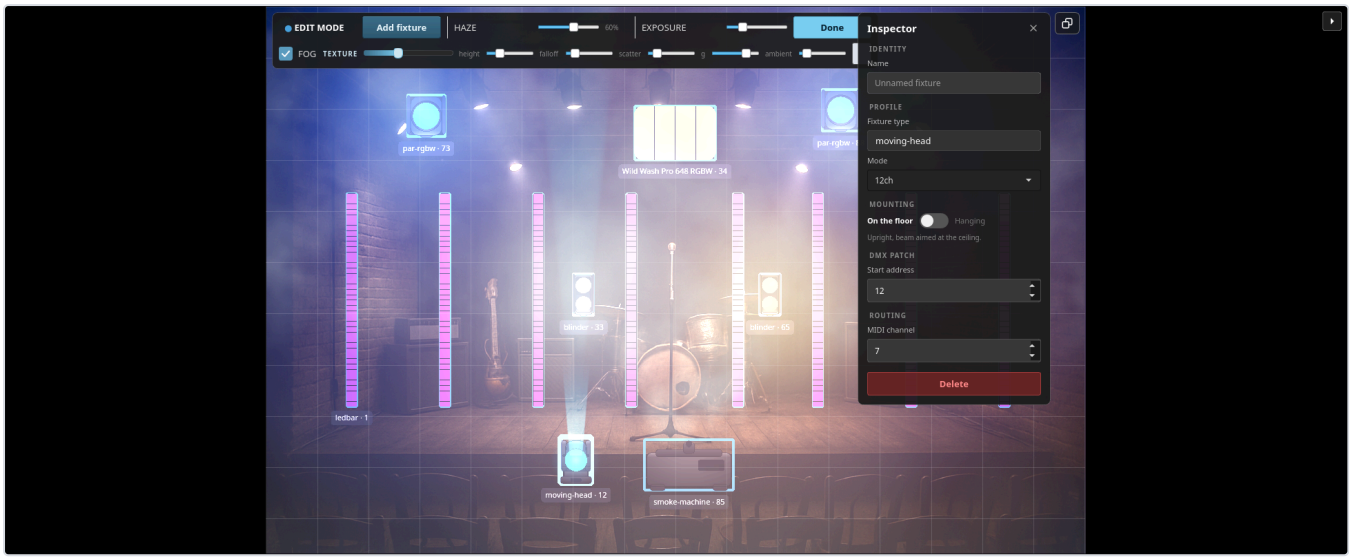


Figure 42 — *The rig patched and placed, with the Inspector open on the moving head.*

Raise the **HAZE** slider to something like 50–60 %. Without haze in the air a beam is invisible, and you will spend an hour wondering why the moving head looks broken. → 4.3

Click **Done**.

Step 5 — Build the looks you will reuse

Before writing any timeline, build the handful of looks the show keeps coming back to — a warm wash, a cold wash, a blinder hit, a beam look.

Open **Add fixture** again and go to the **Scenes** tab (you can cancel out of the dialog afterwards; this tab writes on its own). Dial a look on the faders, watch it in **SIMULATED OUTPUT**, name it, press **Create**.

Tag scenes with song parts (CHORUS, VERSE…) if you want the app to be able to offer them automatically.

> Remember that scenes belong to the fixture **type** and are shared by every show that uses it — and that a saved song addresses a scene by its position in the bank, so add new ones at the end. → 5.4

Step 6 — Write the lighting

Select a song, then start dropping cues from the **Library** onto the fixture lanes.

- **Click a card first** to audition it. The stage shows you exactly what it does before it becomes a clip.
- **Drag it onto the lane** of the fixture it belongs to. The cursor tells you when a drop makes no sense on that lane.
- Drag **song parts** onto the *waveform*, not onto a lane — they mark the arrangement.

Then shape what you dropped: drag clips to time, drag their edges to length, drag the gold triangles for fades. Nothing is committed until you release the mouse, and **Ctrl+Z** undoes anything you regret. → 6.7, 6.6

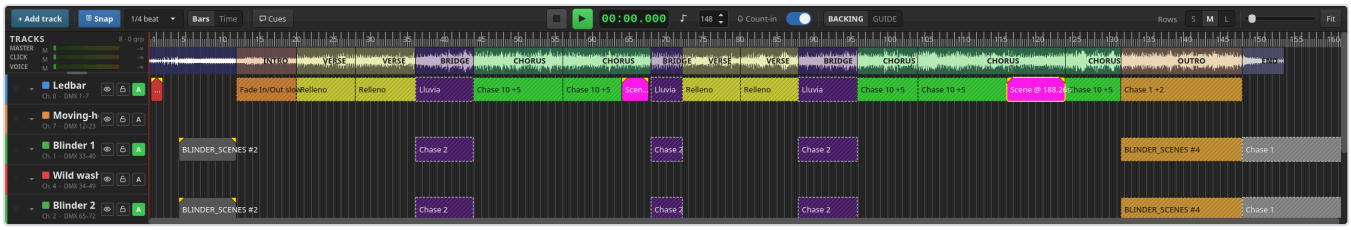


Figure 43 — *Clips written across the fixture lanes, with the arrangement marked on the waveform.*

Double-click any clip to open its editor and set its parameter precisely — a blend time, a chase, an exact channel value. → 6.9

> Two habits that save time later: **use the lock button** on lanes you have finished so a stray drag cannot disturb them, and remember that **deleting a clip leaves a blackout** in its place, which is usually what you want.

Step 7 — Count-ins and cues for the band

Turn the **Count-in** switch on for songs that start cold, and off for songs that segue. Set how many bars — and whether the last one is counted aloud — once, in **Settings ▸ Audio**.

Add spoken cues with the **Cues** button for anything the band needs warning about (*"entra batería"*, *"solo"*). They play on the voice bus, which in split routing goes to the in-ears and not to the room. → 6.1.3, 6.1.4

Step 8 — Set up the audio for the stage

Settings ▸ Audio: pick the interface, and choose a **Split** routing so the backing goes to the PA on one channel and the click plus voice go to the in-ears on the other. Check the balance with the **MASTER**, **CLICK** and **VOICE** sliders in the timeline's header column. → 9.3

Step 9 — Route the lights

At home, leave **Quick mode** on **All software**: the internal controller runs and feeds the virtual stage, and you need nothing plugged in.

At the venue, switch to **Software + real lights** and fill in the Art-Net address of your node (or tick **USB DMX** and pick the interface). The signal flow panel on the right should read **✓ Routing valid — no conflicts**.

Check the **DMX Channels** tab: if the numbers are moving there, the application is producing DMX and anything still dark is downstream. → 8.1, 8.2

Step 10 — Set up the tablet

Settings ▸ Web Remote: switch it on, note the address, open it on the tablet.

Check the sync pill says **SYNCED**, run a song, and confirm the setlist, the section countdown and the cue buttons all do what you expect from the far side of the stage. → 10

Step 11 — Rehearse, then pin a snapshot

Run the set. Fix what looks wrong.

When it is right, **VERSIONS ▸ Snapshot** and name it (`before Billares`). A named snapshot is pinned and never pruned, so whatever happens during load-in you have a known-good state one click away — and restoring it snapshots the current state first, so even the rollback is reversible. → 6.4

Step 12 — Take it to the gig

SHOW ▸ Export writes a single `.showpack` with the audio, the MIDI, the patch and the fixture definitions for every song. Copy it to the machine that will run the show and use **SHOW ▸ … ▸ Import bundle…**.

If the show runs from a rack box with no screen, start it with `--headless` and drive the whole night from the tablet. → 6.1, 10.4

Quick reference

I want to…	Go to
Add a light	**Edit stage** ▸ **Add fixture**, or **+ Add track** on the timeline
Move a light on stage	**Edit stage**, then drag it
Change a light's address or channel	**Edit stage**, select it, use the Inspector
Make a beam visible	Raise **HAZE** on the edit toolbar
Save a look for reuse	**Add fixture** ▸ **Scenes** tab, or **Save as scene…** in a scene clip
Try a cue without committing	**Click** it in the Library
Add a cue to a song	**Drag** it from the Library onto the fixture's lane
Undo	**Ctrl + Z**
Really remove a clip	Delete it, then right-click the blackout ▸ **Remove blackout**
Send lights to a real rig	**Settings** ▸ **Input / Output** ▸ **Software + real lights**
Send the click to the in-ears only	**Settings** ▸ **Audio** ▸ **Routing** ▸ **Split**
Run the show from a tablet	**Settings** ▸ **Web Remote**
Go back to yesterday	**VERSIONS**, pick a snapshot, **Restore**
Move the show to another machine	**SHOW** ▸ **Export**, then **…** ▸ **Import bundle…**
Start over with settings	Delete `userprefs.json` — shows are not affected

Appendix A. MIDI note reference

Every clip in this application is a MIDI note, and the same three-part rule decides what it does:

> The **note number** is a *verb*. The **MIDI channel** selects *which fixture* the verb acts on. The **velocity** is that verb's *parameter*.

You do not need this table to use the timeline — the clip editor names every verb in words and gives each one its own control. It is here for the cases where you are looking at raw MIDI: authoring a show in another sequencer, driving the lighting from BBeat in mode 1, or working out why a note imported from somewhere else does nothing.

A.1 The full layout

Scope says whether the MIDI channel matters. **Global** verbs are dispatched before the fixture is looked up, so they act on the whole rig whatever channel they arrive on. **Fixture** verbs do nothing at all if no patched fixture owns that channel.

Note	Verb	Scope	Velocity means
0	*reserved*	—	—
1 – 23	Set DMX channel *n* of this fixture	fixture	**DMX value = velocity × 2**
24 – 35	Load scene *n* − 24 of the fixture's **current** bank	fixture	blend time in units of 100 ms
36 – 47	Select bank *n* − 36	fixture	ignored
48	Take manual control	fixture	ignored
49	Manual — hold the current frame	fixture	ignored
50	Black out this fixture	fixture	ignored
51	**Black out everything** — a toggle	**global**	ignored
52	Blend time	fixture	milliseconds = velocity × 100
53	Colour-change time	fixture	milliseconds = velocity × 100
54	Chase-change time	fixture	milliseconds = velocity × 100
55	Palette random **on**	fixture	ignored
56	Palette random **off**	fixture	ignored
57	Previous scene in the current bank	fixture	blend time × 100 ms
58	Next scene in the current bank	fixture	blend time × 100 ms
59	Load bank **and** scene together	fixture	**bank = velocity ÷ 8, scene = velocity mod 8**
60	**START** — build the colour palette	fixture	**hue = velocity ÷ 6, mode = velocity mod 6**
61 – 69	Song part — INTRO … END	**global**	ignored
70	Load chase	fixture	chase index
71	Track setup flags — never produces light	fixture	bit 0 auto, bit 1 mute, bit 2 lock
72 – 75	Colour macro — WHITE / ORANGE / CYAN / PINK	fixture	ignored
76 – 127	*reserved*	—	—

A.2 Song parts (notes 61 – 69)

A song-part marker does not set a look directly. It tells every fixture that is in **auto** mode to load whatever scene or chase its own definition has tagged for that part; fixtures with nothing tagged for it are blacked out. That is why the same marker produces a different look on every fixture, and why tagging scenes in the fixture Scenes tab (5.4) is what makes the automatic program work at all.

Note	Part	Note	Part	Note	Part
61	INTRO	64	BREAK	67	SOLO
62	VERSE	65	BRIDGE	68	OUTRO
63	CHORUS	66	INST	69	END

These are the markers you drag onto the **waveform**, not onto a fixture lane — they belong to the song rather than to a light.

A.3 Colour macros (notes 72 – 75)

Note	Macro	RGB
72	WHITE	`255, 255, 230`
73	ORANGE	`255, 165, 0`
74	CYAN	`0, 255, 255`
75	PINK	`255, 192, 203`

The colour is fitted to whatever the target fixture can actually mix, so a macro sent to a fixture with no colour channels does nothing rather than something wrong.

A.4 The START palette (note 60)

Note 60 packs two choices into one velocity:

```
hue_index    = velocity ÷ 6      # 0–11 pick a fixed hue; 12 and up = random
palette_mode = velocity mod 6    # see the table below
```

Value	Palette mode	What it builds
0	COMPLEMENTARY	the hue and its opposite
1	ANALOGOUS	the hue and its two neighbours
2	TRIADIC	three hues evenly spaced around the wheel
3	COLOR+WHITE	the hue alternating with white
4	MONOCHROMATIC	one hue at several lightnesses
5	SINGLE	the hue alone

So velocity 14 is hue 2 in TRIADIC mode; velocity 75 is a random hue in MONOCHROMATIC. The timeline's own **Edit START Event** dialog (6.9) is a picker over exactly this arithmetic.

Setting a palette does **not** start it cycling. It stores the palette on the fixture; a song-part marker is what arms it.

A.5 Things that catch people out

A velocity of 0 is a note-off. That is MIDI, not a choice this application made. A verb whose parameter is genuinely zero — a channel at 0, a blend of 0 ms — cannot be expressed as a note-on with velocity 0. Scene loads work around it by clamping the blend to at least 100 ms; for a channel, send the *blackout* verb (note 50) rather than a zero.

Channel values are doubled, so only even values are reachable. Velocity 100 means DMX 200. An odd DMX value cannot be sent from a clip; the clip editor says so when you try, and shows what will actually be replayed.

Only a fixture's first 23 DMX slots can be driven this way, because note 24 is already a scene verb. Channels past that are still patched, still rendered and still reachable from the fixture's stored scenes — they just cannot be addressed clip by clip. → 6.2

Notes 24 – 35 depend on the current bank. They load a scene *by position in whichever bank the fixture is currently on*, so a bank-select verb (36 – 47) earlier on the lane decides what they mean. Note 59 avoids the ordering problem by carrying both, at the cost of reaching only the first eight scenes of a bank.

Scene numbering is positional. Inserting, deleting or reordering scenes in a bank shifts every note after it, and a saved song will load a different look. Renaming is always safe. → 5.4

Note 71 never makes light. It carries a lane's own settings — auto, mute, lock — so they survive in the `.mid`. The runtime ignores it and the timeline does not draw it as a clip.

A.6 One universal layout, many fixtures

Nothing above names a fixture type. A verb resolves against whatever definition the addressed fixture has: "channel 3" is that fixture's third DMX slot, "scene 2" is the second scene of its current bank, "chase 4" its fifth chase. The one layout works for a six-channel PAR and a thirty-two-channel moving head without a special case for either.

That is also what makes mode 1 work: the hardware controller reads the same fixture definitions and dispatches the same notes, so a show designed against the simulator plays back identically on the real rig.

Appendix B. Building a rack image with Buildroot

Mode 3 puts the application on a small board in the rack with no screen, driven entirely from a tablet. This appendix builds the operating system for that board: a single bootable image that comes up straight into `main.py --headless`, with the web remote as the only interface.

There is no login, no desktop and no package manager on the finished box. It boots, it plays the show, and it serves the remote.

> **The shipped configuration targets x86_64 with EFI firmware** — a mini-PC or NUC-class board, not a Raspberry Pi. See B.10 before pointing this at an ARM board.

B.1 What you need

Build host	Linux, x86_64, with a working build toolchain (`make` , `gcc` , `git` , `bash`)
Disk	~30 GB free for the Buildroot tree and output
Time	1–3 hours for the first build; seconds to minutes afterwards
Network	Only on the **build host**, and only for the first build
Target	An x86_64 board that boots EFI, and something to write the image to

The finished box needs no network at the venue. Every Python wheel is resolved and installed on the build host, which is the whole reason the image is as large as it is.

B.2 Lay the two trees out side by side

The configuration reaches into this repository with relative paths, so Buildroot must be checked out **next to it**:

```
Projects/
├── buildroot/                # the Buildroot tree
└── tupperDMX-simulator/      # this repository
```

Nothing is copied into the Buildroot tree. `BR2_ROOTFS_OVERLAY` and the second `BR2_ROOTFS_POST_BUILD_SCRIPT` both point back here through `$(TOPDIR)/../tupperDMX-simulator/`, so the app, the init script and the Samba configuration are always read from your working copy.

B.3 Build

```
cd ../buildroot
cp ../tupperDMX-simulator/buildroot/x86_config .config
make olddefconfig
make
```

The result is **`output/images/disk.img`** — a complete disk with its EFI partition and a 3 GB root filesystem.

The first build downloads roughly 900 MB of Python wheels on the build host. Later builds hit uv's cache and finish in seconds.

B.4 Write it to the target

`disk.img` is a whole-disk image, so it goes to the device, not to a partition:

```
sudo dd if=output/images/disk.img of=/dev/sdX bs=4M status=progress conv=fsync
```

> Check `/dev/sdX` twice. `dd` will overwrite whatever you point it at, without asking, including the disk you are working from.

Then boot the target from it with EFI (not legacy/CSM) enabled.

B.5 What ends up on the box

Path	What
`/opt/tupperdmx/`	the application source (no `tests/`, `docs/`, `.venv`, `.git`)
`/opt/tupperdmx/python/`	a relocatable CPython 3.13 with every locked dependency (~900 MB)
`/usr/bin/uv`, `/usr/bin/uvx`	a static build, for maintenance on the box
`/var/lib/tupperdmx/`	the service's `HOME` and working directory
`/etc/init.d/S99tupperdmx`	the init script
`/mnt/tupperdmx/`	the file share — a bind mount of the whole data directory
`/etc/samba/smb.conf`	the share definition

Everything the application writes lands under `/var/lib/tupperdmx`: `logs/`, `.config/tupperDMX/userprefs.json`, and the show projects in `Documents/TupperDMX/`. Back up that one directory and you have the box — and since that directory *is* the file share (B.6), backing it up means dragging one folder off the network.

Why it carries its own Python

The build fetches CPython 3.13 rather than using the target's own interpreter: Buildroot ships 3.14, and several dependencies publish no wheel for it yet. The wheels are resolved on the build host — host and target are both x86_64/glibc, so they are the same wheels the target would have downloaded — which is what lets the box boot with no network at the venue.

B.6 The file share

The box exports `/mnt/tupperdmx` over SMB, with no accounts and no password:

```
\\TUPPERDMX\tupperdmx          # Windows
smb://tupperdmx/tupperdmx      # macOS, GNOME, KDE
smbclient //tupperdmx/tupperdmx -N
```

That share is the application's entire data directory. It is a bind mount of `/var/lib/tupperdmx`, so the share root is everything the box writes — no copying, no sync, one set of bytes under two names:

```
<share>/Documents/TupperDMX/*.show/       the show projects
<share>/logs/tupperDMX_YYYYMMDD.log       the running log
<share>/.config/tupperDMX/userprefs.json  this machine's preferences
<share>/boot.log                          whatever died before logging came up
```

That covers the three things you would otherwise need a screen and a keyboard for:

Loading a show. Copy a `.show` folder into `Documents/TupperDMX/` and it lands exactly where the application looks for it — no import step. Restart the service and the show is there. (Or export a `.showpack` bundle from the desktop application, 7.1, and drop that in instead.)

Reading the log when something did not behave at the venue: it is right there, live, while the box is running.

Editing the preferences — the audio device, the I/O routing, the web-remote port — by hand, for a box that has no Settings dialog to open.

> **Stop the service before editing `userprefs.json`.** It is read once at startup, so an edit changes nothing until a restart — and the application rewrites the file whenever it saves, which will overwrite your edit from underneath you. `/etc/init.d/S99tupperdmx stop`, edit, start.

Two more things worth knowing:

- **Dot files are deliberately visible.** Samba hides them by default, which would put `.config/tupperDMX/userprefs.json` out of reach of exactly the clients this share exists for.
- Anything written into `/mnt/tupperdmx` while the bind mount is *not* active is shadowed once it is — a mount hides the underlying directory rather than merging with it.

> **Windows 10 and 11 refuse guest logons over SMB by default.** Reaching the share from Windows needs *Enable insecure guest logons* (`gpedit.msc` → Computer Configuration → Administrative Templates → Network → Lanman Workstation). Linux and macOS clients connect as-is.

B.7 Running it

```
/etc/init.d/S99tupperdmx {start|stop|restart|status}
```

`stop` sends SIGTERM and waits up to 20 seconds; the application catches it and runs the same shutdown path as closing the window, which flushes the pending MIDI autosave. **Stop the service rather than pulling the power** if you have been editing.

Start-up ordering matters and is already handled: the service runs last, after DHCP has an address to serve the remote on and after the audio server is up.

Open the remote from a tablet on the same network — the address is the box's IP on the configured port. → 10

B.8 USB DMX and USB MIDI

Buildroot's own kernel configuration for this board is 90 lines — USB storage and onboard audio, and nothing else this application needs. A **kernel fragment** (`buildroot/linux.fragment`) is merged on top of it, leaving Buildroot's file untouched and upgradeable, and it is what makes the rack box usable with real gear:

Added	What it gets you
`CONFIG_USB_SERIAL` + `CONFIG_USB_SERIAL_FTDI_SIO`	**USB DMX dongles.** The DMX worker resolves an FTDI serial number to a `/dev/ttyUSB*` port and opens it at 250000 baud, 8N2 — DMX512 on the wire. Without this the dongle never appears as a serial port and every device fails with *"No FTDI serial port found"*
`CONFIG_USB_SERIAL_CP210X`, `_PL2303`, `_CH341`, `CONFIG_USB_ACM`	the other bridge chips found on serial gear, and Arduino-class boards
`CONFIG_SND_SEQUENCER`	**MIDI at all.** The MIDI layer binds to the ALSA *sequencer*; without it the box has no MIDI ports to offer
`CONFIG_SND_USB_AUDIO`	**USB MIDI controllers and USB audio interfaces.** The stock configuration only opens the menu gate, not the driver

So a rack box built from this configuration will enumerate a USB DMX interface and a USB MIDI controller, and can use a USB audio interface instead of the board's onboard output.

> If you edit the fragment, run `make linux-reconfigure` before `make`. Buildroot does not notice the file changing on its own, and you will get a kernel built from the previous contents with no warning.

B.9 Changing the configuration

```
cd ../buildroot
make menuconfig
make
cp .config ../tupperDMX-simulator/buildroot/x86_config   # carry it back
```

Copy the config back into this repository, or the next person to build gets your old one.

> **Enabling a package can break the next build, somewhere unrelated.** Buildroot never re-runs a package that has already been built, *even when its options changed*. So enabling something that pulls in a new sub-option of an already-built package leaves that package stale, and the failure surfaces elsewhere entirely.
>
> After enabling a package, check what its `Config.in` selects, and `make <pkg>-dirclean` anything on that list that was already built. Use `-dirclean`, not `-rebuild`: the configure options changed, so the tree has to be re-extracted and re-configured.

The configuration also carries the runtime library closure that Qt needs but that the application's dependency list never mentions — a C++ toolchain and OpenMP for the wheels, Mesa and the X11 libraries because `libQt6Gui` links `libGL` *even under the offscreen platform*, PulseAudio because `libQt6Multimedia` links `libpulse`, Kerberos because `libQt6Network` links it, plus fontconfig, freetype and the compression libraries. Removing any of them stops the box booting into the application, with a loader error rather than an obvious one.

B.10 Other boards

The shipped `x86_config` is derived from Buildroot's `pc_x86_64_efi_defconfig`, and the build assumes the host and the target share an architecture — that is what makes it safe to resolve the target's Python wheels on the build machine.

Targeting an ARM board (a Raspberry Pi, say) therefore needs more than a different defconfig: the wheel set has to be resolved for that architecture, and the static `uv` fetched for it. The overlay — the init script and the Samba configuration — carries over unchanged.

B.11 Known gaps

These are real limitations of the current image, not warnings about misuse.

- **The box trusts its network completely.** The root password is still Buildroot's default and sshd is enabled; the SMB share has no accounts and no password — and it now exposes the preferences and the logs alongside the shows; the web remote has no authentication. Keep it on the stage network, and do not put it on the venue's guest Wi-Fi.
- **No text-to-speech voice is preloaded**, so the spoken count-in is unavailable until the box downloads one — which it cannot do without a network. The log says `Spoken count-in unavailable`. The click

and the count-in bars still work; only the spoken numbers are missing.